



ค่ายโอลิมปิกวิชาการ วิชาคอมพิวเตอร์

ค่าย 2

1. Linked Lists, Stacks and Queues

รายการโยง สแตค และแถวคอย

ดร.สรร รัตนสัญญา

san.r@sci.kmutnb.ac.th

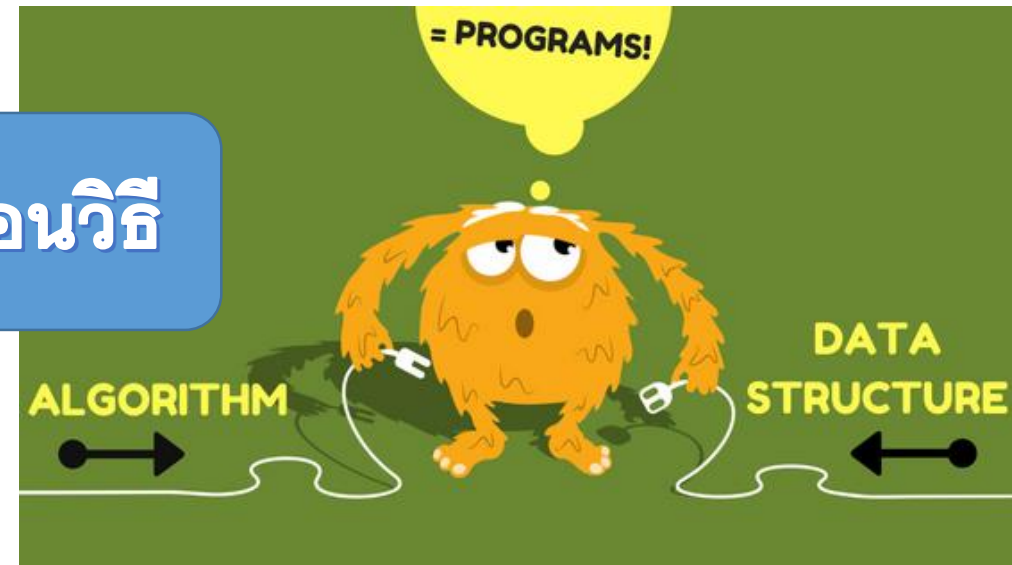
Outline

- Programs vs. Good Programs
- Importance of Data
- Why Data Structures?
- Why Algorithms?
- Array and Memory Map
- Node

โปรแกรม (Program) คือ

- กลุ่มของคำสั่ง เพื่อใช้ในการแก้ปัญหา ปัญหาหนึ่ง

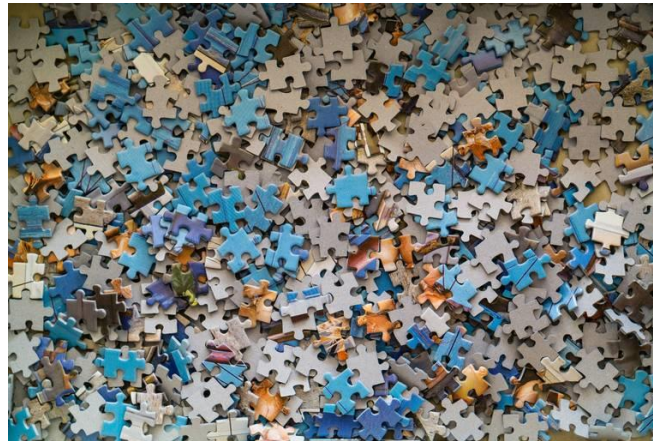
โปรแกรมที่ดี = โครงสร้างข้อมูล + ขั้นตอนวิธี



ตัวอย่างปัญหา

- หารากของสมการ
- ถอดสแควร์รูท
- คำนวณเกรด
- เจียวก๊วย
- ดริปปากาแฟ
- เดินทางมามหาวิทยาลัย
- ซ้อปปิ้ง

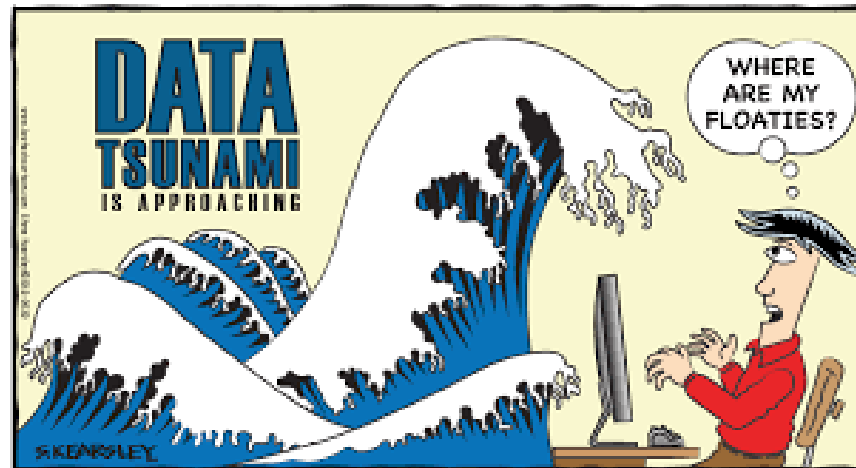
ในการแก้ปัญหา ข้อมูล (Data) คือสิ่งจำเป็น



ข้อมูล (Data)

- ข้อมูล

- ตัวเลข
- ตัวอักษร



- การจัดการข้อมูล

- อ่าน

- อ่าน

- ค้นหา

- เขียน

- เพิ่ม / แทรก

- ลบ

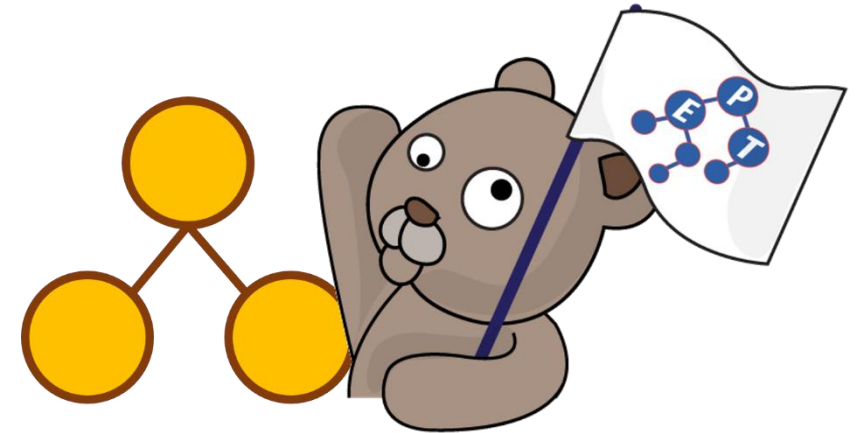
โครงสร้างข้อมูล (Data Structures)

- Abstract Data Type
 - Linked Lists, Stacks, Queues, ...
- การจัดระเบียบข้อมูลให้อยู่ในรูปแบบที่เหมาะสม เพื่อให้สามารถแก้ปัญหาได้อย่างมีประสิทธิภาพ
- ดังนั้นจำเป็นต้องมีวิธีการเพื่อจัดการกับข้อมูลตามโครงสร้างที่ออกแบบไว้โดยเฉพาะ



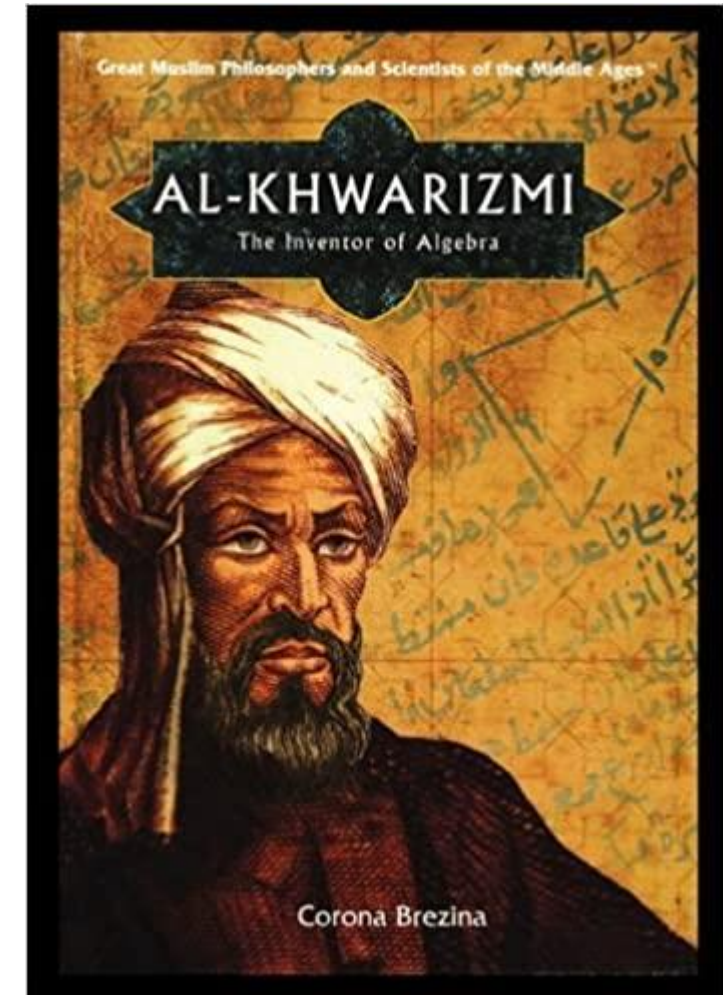
ตัวอย่างโครงสร้างข้อมูล

- ข้อมูลนักศึกษา (รหัส, ชื่อ, เกรดเฉลี่ย)
- ตารางอัตราแลกเปลี่ยนสกุลเงิน
- แผนภูมิตระกูล



ขั้นตอนวิธี (Algorithms)

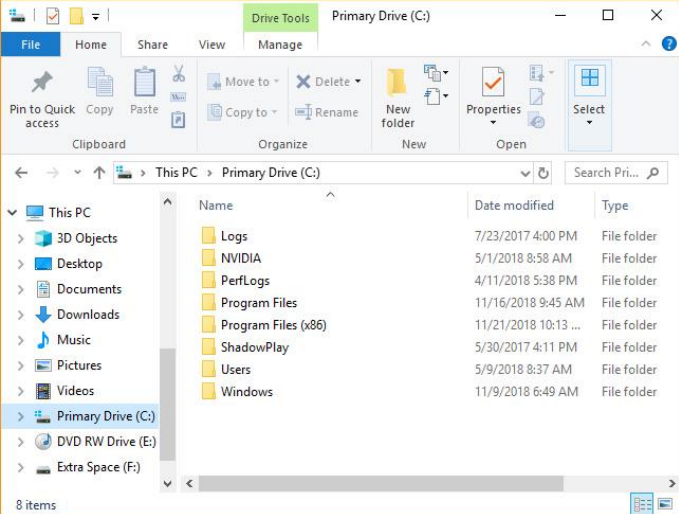
- กระบวนการแก้ปัญหาที่สามารถเข้าใจได้ มีลำดับ หรือ
- วิธีการในการแก้ไขปัญหาใดปัญหาหนึ่งอย่างเป็นขั้นเป็นตอนและชัดเจน



ตัวอย่างขั้นตอนวิธี

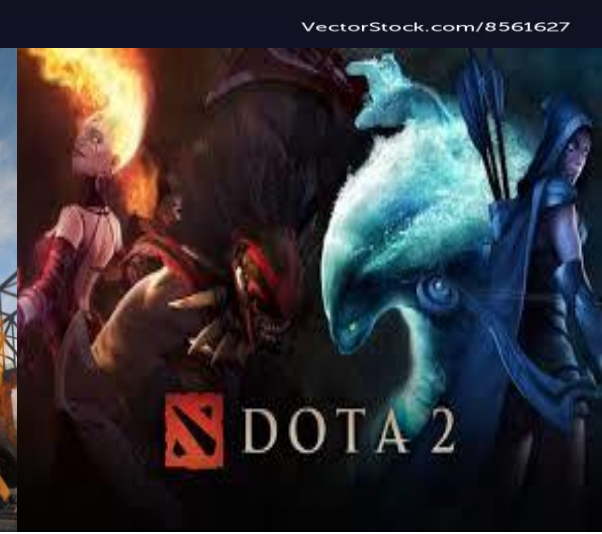
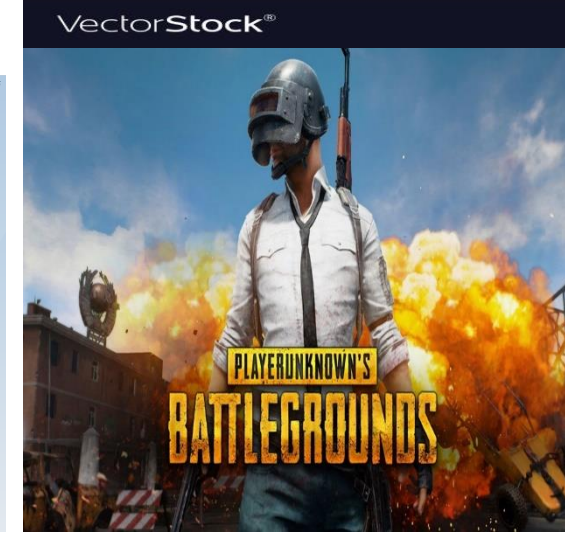
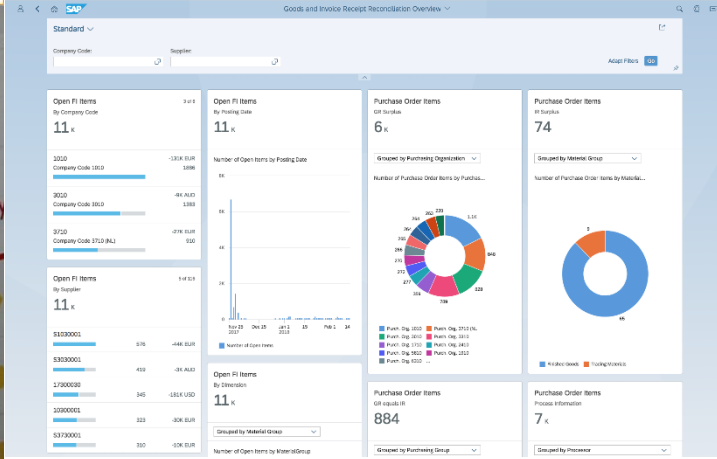
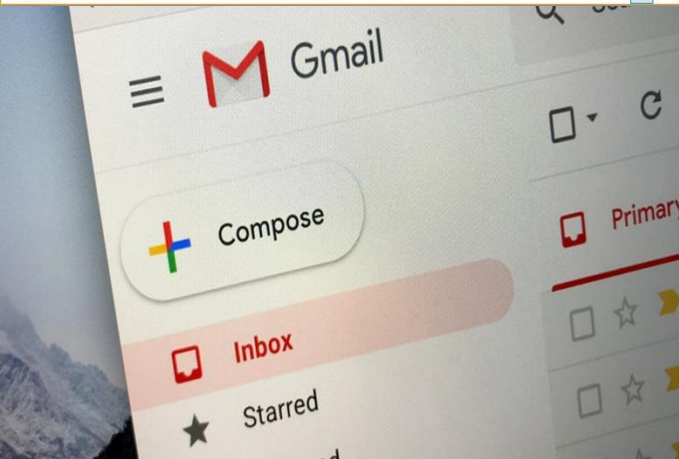
- วิธีการทอนเงิน
- วิธีการใส่ของลงในถุง
- วิธีการทำขนมเค้ก
- วิธีการทำต้มยำกุ้ง





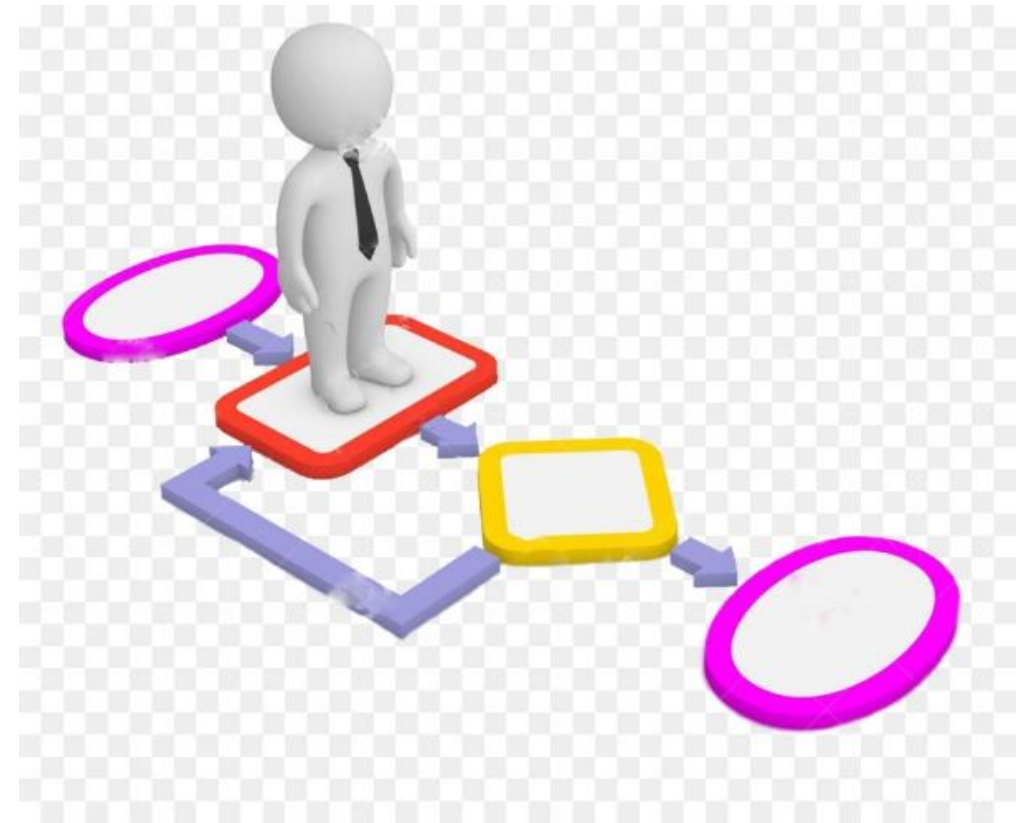
DATA STRUCTURE

MEMORY RETRIEVE
MANAGE COMPLEXITY
LANGUAGE INFORMATION ORGANIZING
INDEX OPERATIONS
EFFICIENT ABSTRACT
STORE TYPES
HASH TABLE
COMPUTER APPLICATION IMPLEMENTATION
DATABASE PERFORM
AMOUNTS
PROCEDURES



Summary

- การดำเนินการกับข้อมูล มี 3 รูปแบบ การค้นหา การเพิ่ม/แทรก การลบ
- โครงสร้างข้อมูล และขั้นตอนวิธีที่เหมาะสมช่วยให้โปรแกรมทำงานได้อย่างมีประสิทธิภาพมากขึ้น
- โครงสร้างข้อมูล คือ การจัดระเบียบข้อมูล
- ขั้นตอนวิธี คือ วิธีการในการแก้ไขปัญหาใดปัญหาหนึ่งอย่างเป็นขั้น เป็นตอนและชัดเจน



Array

- An ordered arrangement
- Contiguous memory

40	55	63	17	22	68	89	97	89
0	1	2	3	4	5	6	7	8

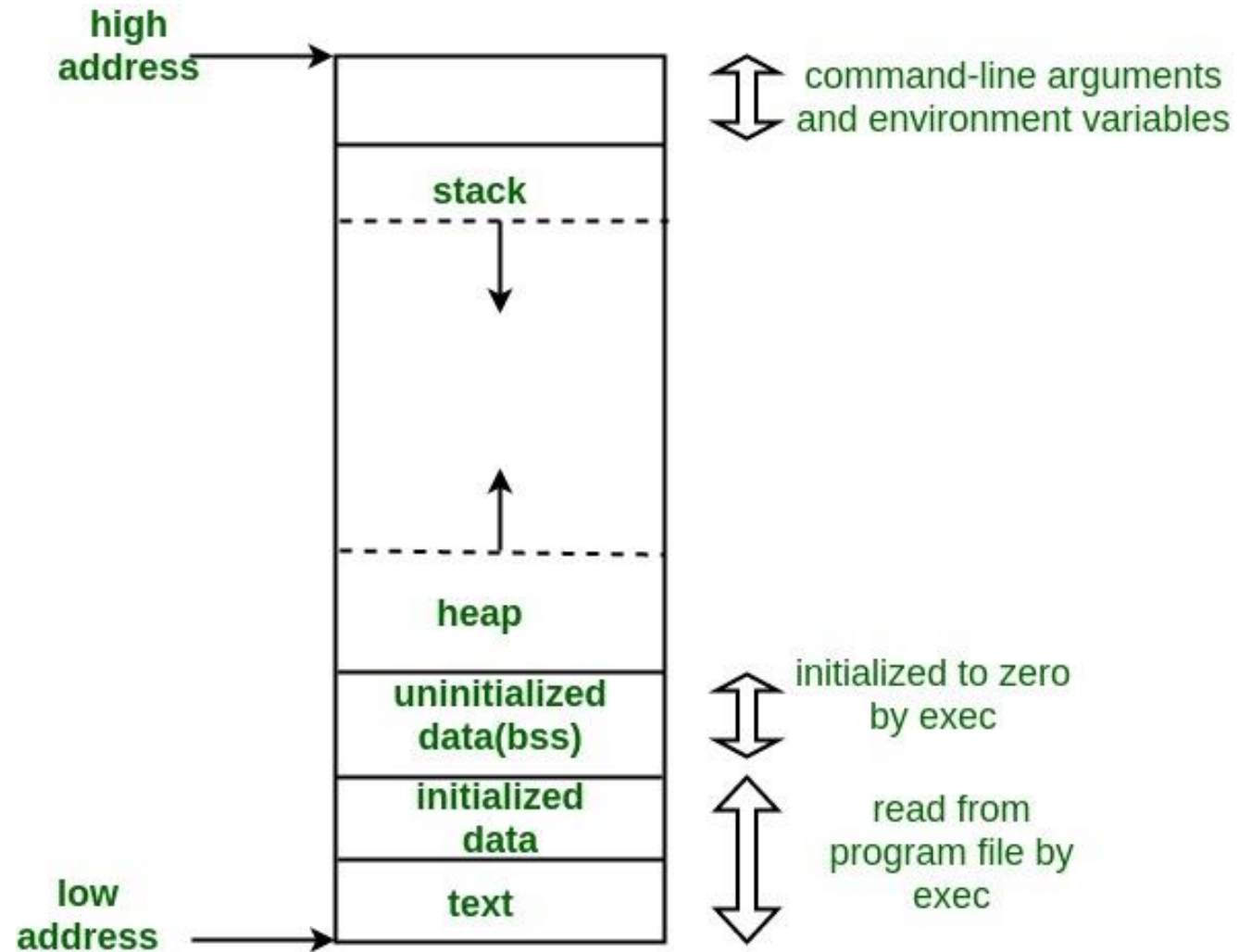
<- Array Indices

Array Length = 9

First Index = 0

Last Index = 8

Memory Map



Pros and Cons of Array

- ถ้ารู้ตำแหน่งจะสามารถเข้าถึงสมาชิกและจัดการข้อมูลได้ทันที เนื่องจากข้อมูลอยู่ติดกัน

- มีขนาดจำกัด
- การแทรก / ลบ ข้อมูล ต้องมีการจัดการที่เหมาะสม
- ต้องการพื้นที่ที่ติดกัน

Performance Analysis of 1-D Array

- $O(1)$

- access, add and search (position given)

- $O(n)$

- insert, delete, add, search

Dynamic Memory Allocation

- malloc, alloc, realloc and free in C
- new and delete in C++
- Allocate on heap area
- programmers need to deal with dynamic allocation themselves

ตัวอย่าง

```
int* ptr;  
  
ptr = (int*)malloc(n * sizeof(int));  
  
ptr = (int*)calloc(n, sizeof(int));  
  
ptr = realloc(ptr, n * sizeof(int));  
free(ptr)
```

C

```
int *ptr = new int(25);  
delete ptr;  
  
int*ptr = new int[n];  
delete [] ptr;
```

C++

Vector in C++

- vector library

- dynamic array

$$\begin{array}{c} 5 \\ \longrightarrow \end{array} + \begin{array}{c} 5 \\ \longrightarrow \end{array} = \begin{array}{c} 10 \\ \longrightarrow \end{array}$$

$$\begin{array}{c} 5 \\ \longrightarrow \end{array} + \begin{array}{c} -5 \\ \longleftarrow \end{array} = 0$$

$$\begin{array}{c} 5 \\ \longrightarrow \end{array} + \begin{array}{c} 10 \\ \longrightarrow \end{array} = \begin{array}{c} 15 \\ \longrightarrow \end{array}$$

$$\begin{array}{c} 5 \\ \longrightarrow \end{array} + \begin{array}{c} -10 \\ \longleftarrow \end{array} = \begin{array}{c} -5 \\ \longleftarrow \end{array}$$

$$\begin{array}{c} 5 \\ \longrightarrow \end{array} + \begin{array}{c} -15 \\ \longleftarrow \end{array} = \begin{array}{c} -10 \\ \longleftarrow \end{array}$$

$$\begin{array}{c} 10 \\ \uparrow \end{array} + \begin{array}{c} -5 \\ \downarrow \end{array} = \begin{array}{c} 5 \\ \uparrow \end{array}$$

ตัวอย่าง

```
#include <iostream>
#include <vector>

using namespace std;

int main()
{
    vector<int> g1;

    for (int i = 1; i <= 5; i++)
        g1.push_back(i);

    cout << "Output of begin and end: ";
    for (vector<int>::iterator i = g1.begin(); i != g1.end(); ++i)
        cout << *i << " ";

    return 0;
}
```

```
Output of begin and end: 1 2 3 4 5
-----
Process exited after 0.1097 seconds with return value 0
Press any key to continue . . .
```

```
for (int i = 0; i != g1.size(); ++i)
    cout << i << " ";
```

Abstract Data Type

- User defined data type
 - structure, union, enum
 - class or object
- Operations to manage that data type
 - read / write
 - misc.



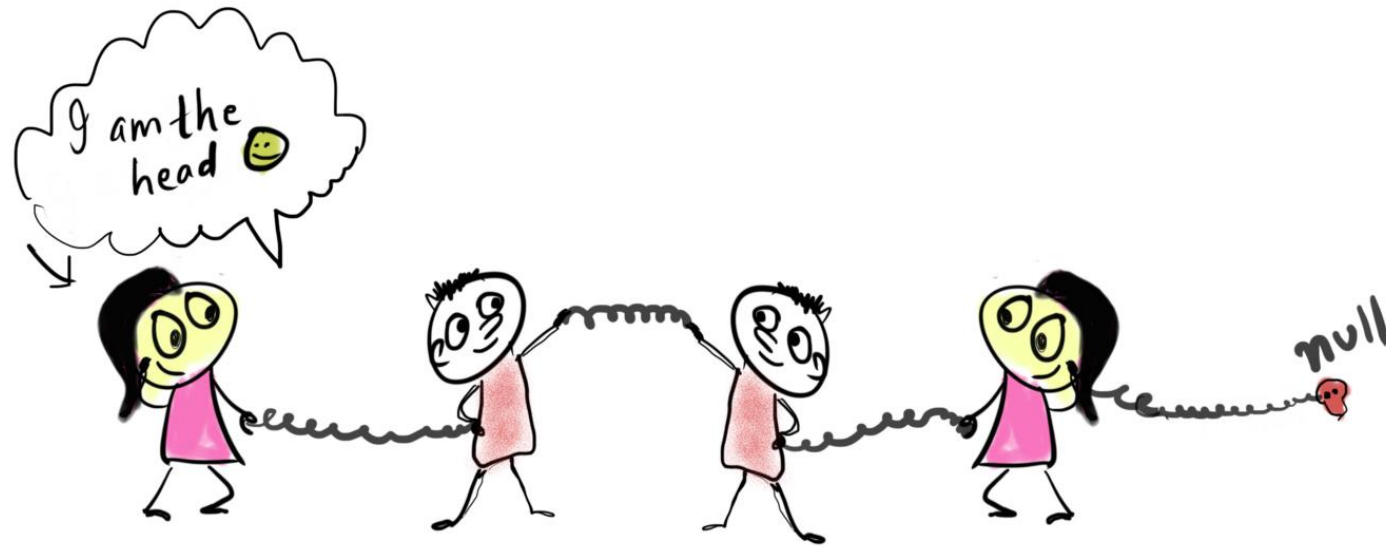
Node

- A backbone of structures we are going to study today



```
class Node {  
    public:  
        int val;  
        Node *next;  
  
        Node(){};  
        Node(int n) {  
            val = n;  
        };  
};
```

รายการ โยง (Linked Lists)

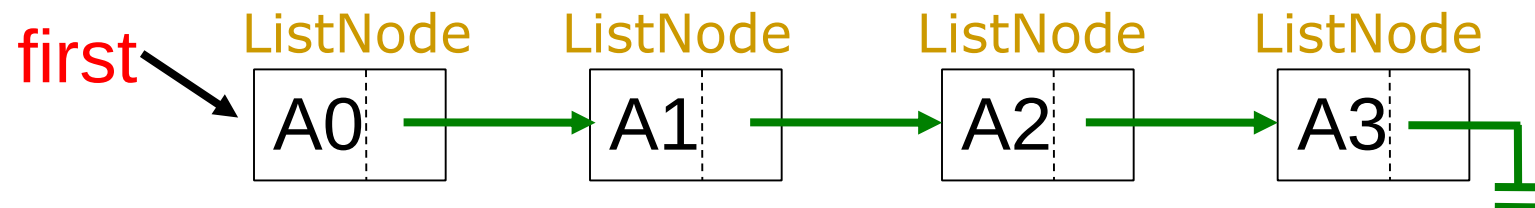


Outline

- Linked list **nodes**
- Linked list operations
 - Insertion
 - Append
 - Deletion
- Linked list representation & implementation
- Other types of linked lists
 - Sorted
 - Doubly-linked
 - Circular

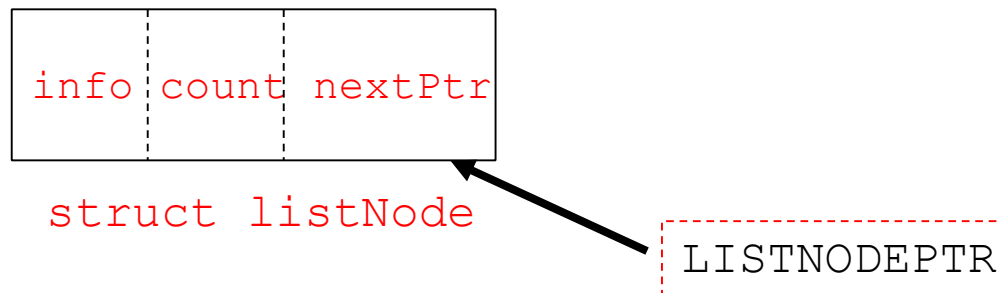
Linked Lists

- รายการโยง ประกอบด้วยกลุ่มของโหนดซึ่งแต่ละโหนดประกอบด้วยเขตข้อมูลอย่างน้อยสองเขต (field)
 - ข้อมูลในโหนด (data)
 - ตัวชี้หรือรายการโยง บอกถึงที่อยู่ของโหนดถัดไปในรายการ (reference to the next Node in the list)
- การเพิ่ม(addition) หรือ การลบ(deletion) ข้อมูลที่อยู่ในรายการใช้เวลา $O(n)$.



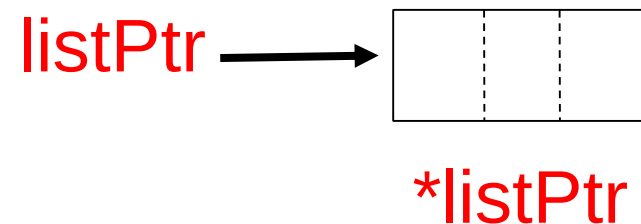
การประกาศรายการโยง(Linked List)ในภาษา C

```
typedef struct listNode LISTNODE;  
typedef LISTNODE *LISTNODEPTR;  
  
struct listNode{  
    char info;  
    int  count;  
    struct listNode *nextPtr;  
};
```



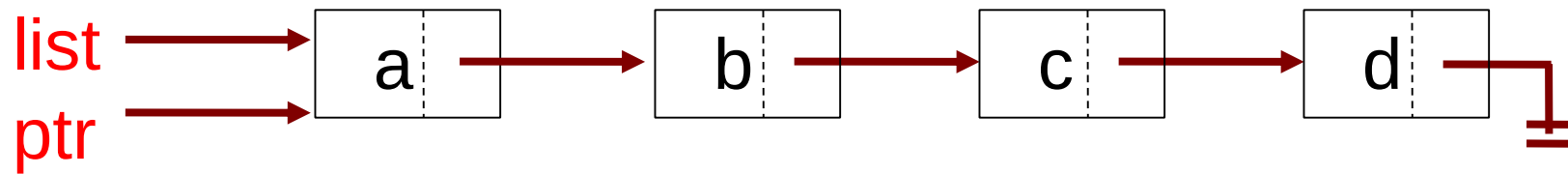
การนำตัวชี้มาใช้งาน

- การเขียนสัญลักษณ์ * ตามด้วยชื่อตัวชี้ จะแทนโหนดของข้อมูลซึ่งตัวชี้ชี้อยู่ เช่น จากการประกาศชนิดตัวชี้ของ listNode การเขียน nextPtr อ้างถึงที่อยู่ของโหนดหนึ่งในลิงค์ลิสต์ ที่ตัวชี้ nextPtr ชี้ไปถึง
- listPtr แตกต่างจาก *listPtr เพราะ listPtr คือตัวชี้ที่อ้างถึงเลขที่อยู่ แต่ *listPtr คือข้อมูลที่ถูกระบุ
- สามารถชี้ไปที่เขตต่าง ๆ ของฟิลด์ได้ โดย ชื่อตัวชี้->ชื่อฟิลด์



การจัดการกับโหนดชนิดตัวชี้

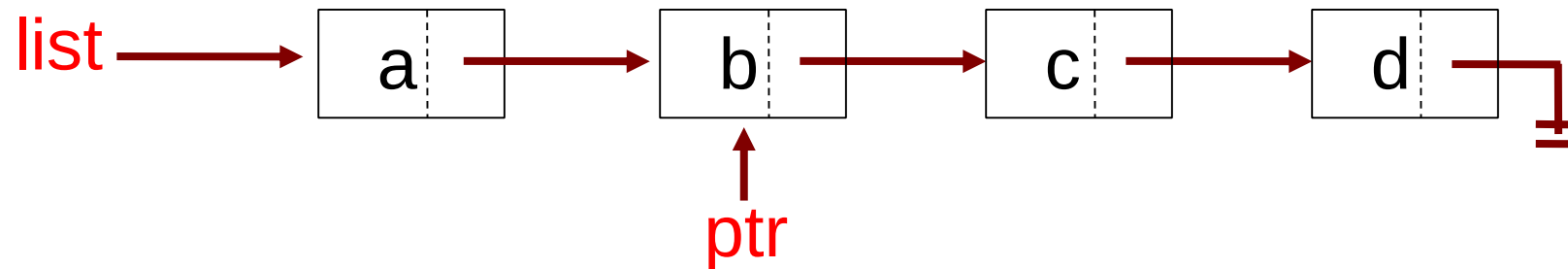
- กำหนดให้ ptr และ ptr->nextPtr เป็นตัวชี้ (Pointer) ชี้ไปที่โครงสร้างข้อมูลที่เป็น listNode



- การสั่งให้ ptr ชี้ไปที่โหนดแรกของรายการ โดย ptr = list;

การจัดการกับ โหนดชนิดตัวชี้ (2)

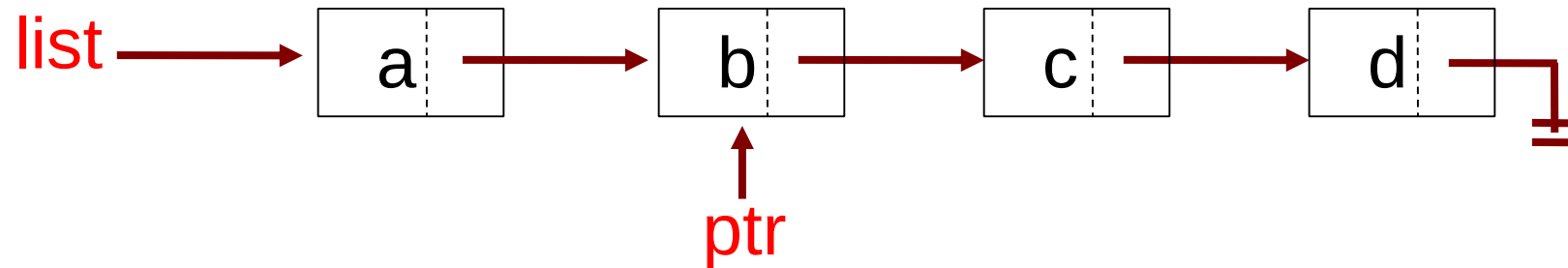
- กำหนดให้ `ptr` และ `ptr->nextPtr` เป็นตัวชี้ (Pointer) ชี้ไปที่โครงสร้างข้อมูลที่เป็น `listNode`



- `list` และ `ptr` เป็นตัวชี้โดย `list` ชี้ไปที่โหนดแรก และ `ptr` ชี้ไปที่โหนดที่สอง

การจัดการกับโหนดชนิดตัวชี้ (3)

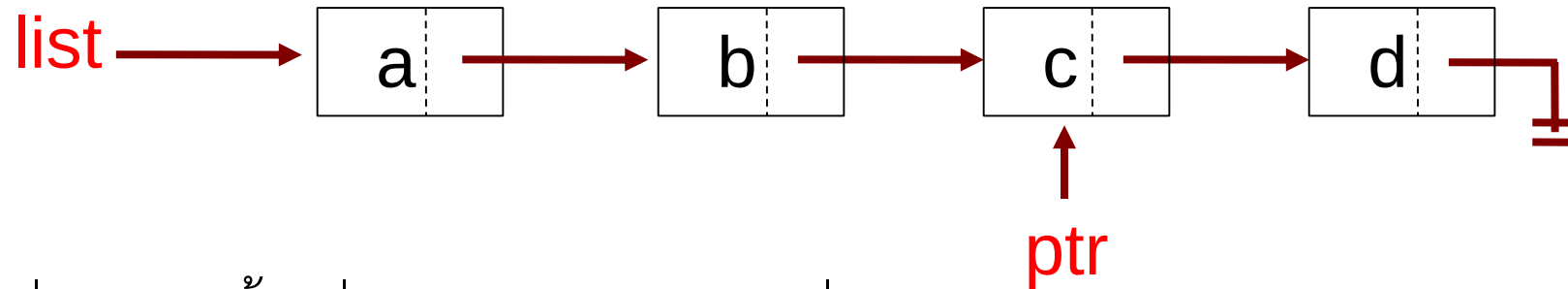
- กำหนดให้ `ptr` และ `ptr->nextPtr` เป็นตัวชี้ (Pointer) ชี้ไปที่โครงสร้างข้อมูลที่เป็น `listNode`



- ให้ `ptr` ชี้ไปที่โหนดที่สองในลิงค์ลิสต์ โดย `ptr=list->nextPtr`

การจัดการกับโหนดชนิดตัวชี้ (4)

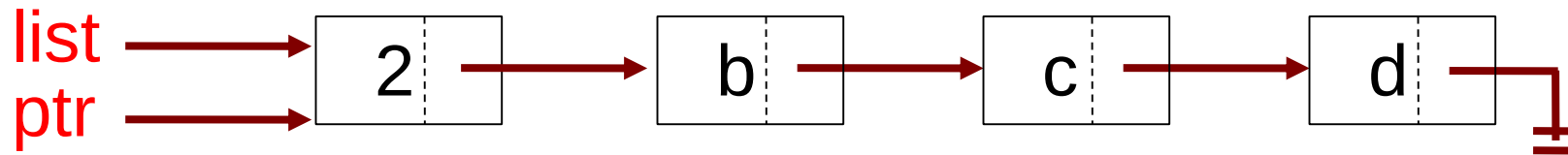
- กำหนดให้ `ptr` และ `ptr->nextPtr` เป็นตัวชี้ (Pointer) ชี้ไปที่โครงสร้างข้อมูลที่เป็น `listNode`



- การสั่งให้ `ptr` ชี้ไปที่โหนดถัดไป โดยใช้คำสั่ง `ptr = ptr->nextPtr;`

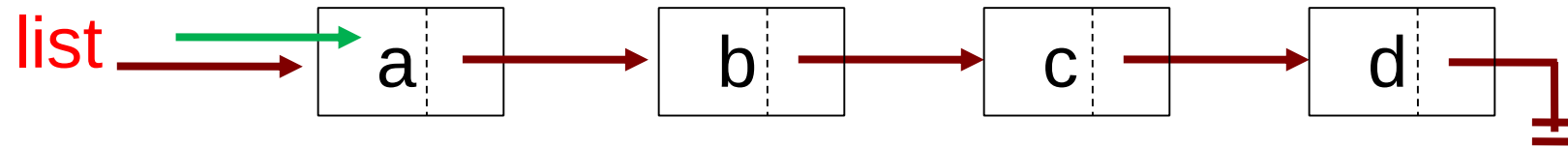
การจัดการกับ โหนดชนิดตัวชี้ (5)

- กำหนดให้ ptr และ ptr->nextPtr เป็นตัวชี้ (Pointer) ชี้ไปที่โครงสร้างข้อมูลที่เป็น listNode

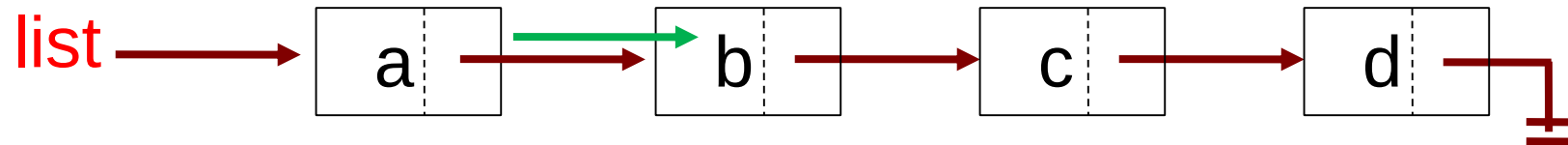


- ให้ค่าของเขต info ในโหนดแรกของลิงค์ลิสต์มีค่าเท่ากับ 2 list->info = 2;

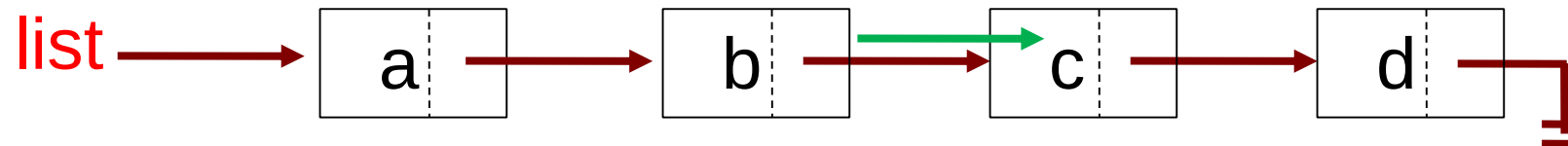
การเรียกใช้ตัวชี้ที่ตำแหน่งต่าง ๆ ในลิงค์ลิสต์



- list ชี้ไปที่โหนดแรกของรายการ



- list->next ชี้ไปที่โหนดที่ 2 ของรายการ



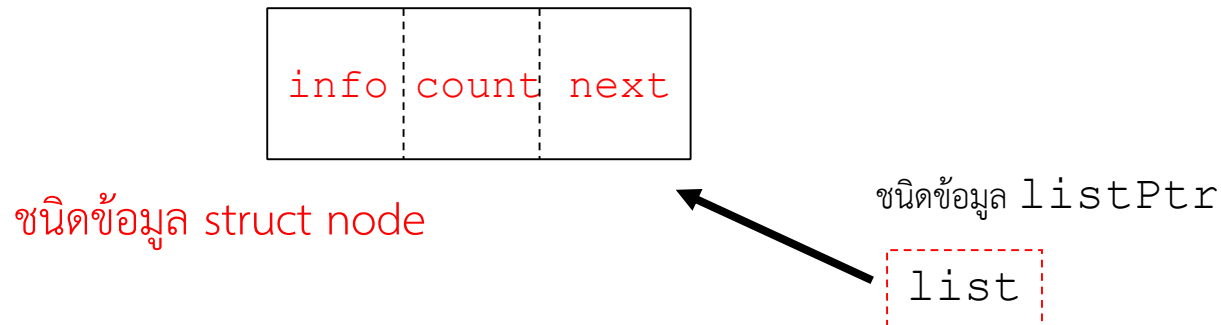
- list->next->next ชี้ไปที่โหนดที่ 3 ของรายการ

การทำงานของลิงค์ลิสต์

```
typedef struct node *ptrNode;
typedef ptrNode listPtr;
typedef ptrNode position;

struct node{
    int info;
    int count;
    position next;
};
```

- ในการค้นหา อ่าน และการพิมพ์ ข้อมูลจากลิงค์ลิสต์สามารถทำได้ โดยการท่องเที่ยวในลิงค์ลิสต์ โดยเริ่มต้นจากกำหนด ptr ที่จุดเริ่มต้นของรายการด้วยคำสั่ง ptr=list
- ต่อมาสามารถเลื่อนตัวชี้ไปที่โหนดถัดไปโดย ptr=ptr->next



การสร้างลิงค์ลิสต์

```
void createList(){
    listPtr list;
    list=malloc(sizeof(struct node));
    if (list){
        list-> info =0;
        list-> count=0;
        list-> next=NULL;
    }
}
```

■ การสร้างลิงค์ลิสต์

■ การทดสอบว่าลิงค์ลิสต์ว่างหรือไม่

```
int isEmpty(listPtr list){
    return (list->count==0);
}
```

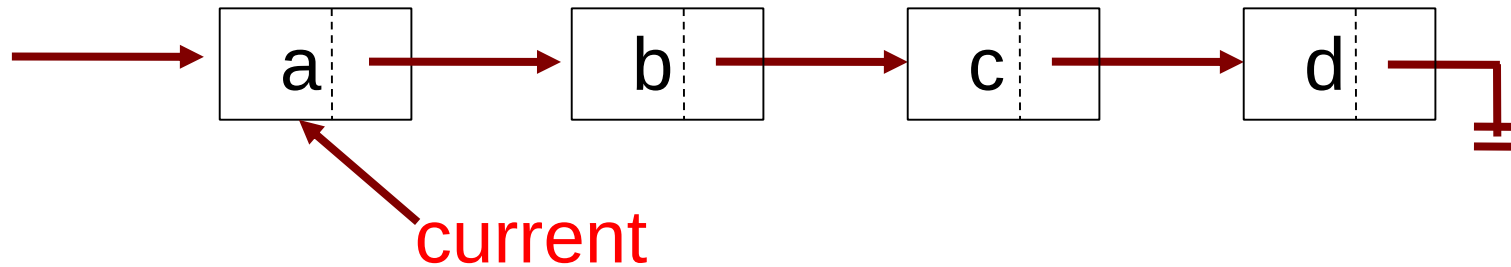
การสร้างและการทำลายโหนด

- การทำงานเกี่ยวกับแถวลำดับหรือระเบียบ เมื่อผู้ใช้ประกาศชื่อโหนดในโปรแกรมแล้วนำโปรแกรมไปแปลชุดคำสั่ง เครื่องคอมพิวเตอร์จะกำหนดเนื้อที่ในหน่วยความจำให้โหนด แต่สำหรับโหนดตัวชี้ซึ่งทำงานแบบพลวัต ภาษาซีมีฟังก์ชัน malloc เพื่อจัดเนื้อที่ในหน่วยความจำสำหรับโหนดชนิดนี้
- ฟังก์ชัน malloc ต้องการพารามิเตอร์ 1 ตัวที่เป็นจำนวนไบต์ที่ต้องการจองเนื้อที่
- รูปแบบ
 - malloc(จำนวนไบต์ที่ต้องการจอง)
 - เช่น malloc(sizeof(x)) เมื่อ x เป็นชื่อโหนด

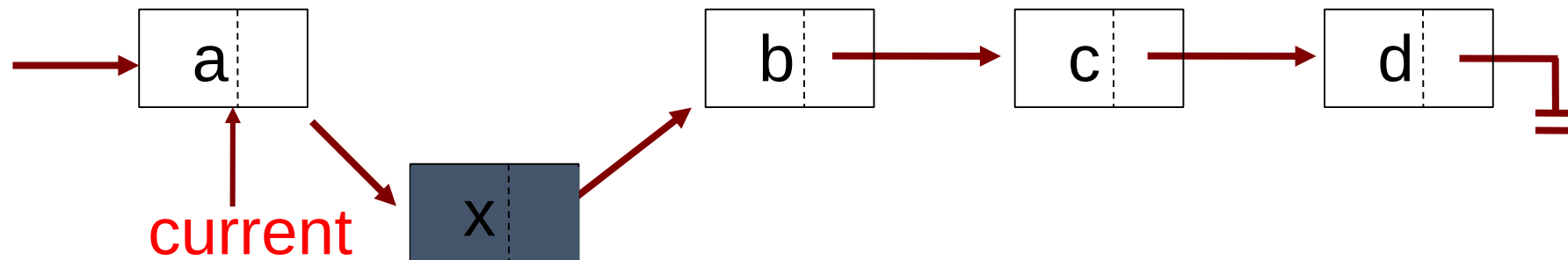
การสร้างและการทำลายโหนด

- เมื่อผู้ใช้ไม่ต้องการใช้ตัวแปรนั้นอีกต่อไปสามารถยกเลิกเนื้อที่สำหรับตัวแปรนั้นเพื่อให้ส่วนอื่นนำไปใช้ได้
- ฟังก์ชันที่ใช้คือ free
- รูปแบบ free(ptr);
 - ptr คือ โหนดชนิดตัวชี้ซึ่งชี้ไปที่หน่วยความจำที่ต้องการยกเลิกการใช้งาน

การเพิ่ม โหนดในลิงค์ลิสต์



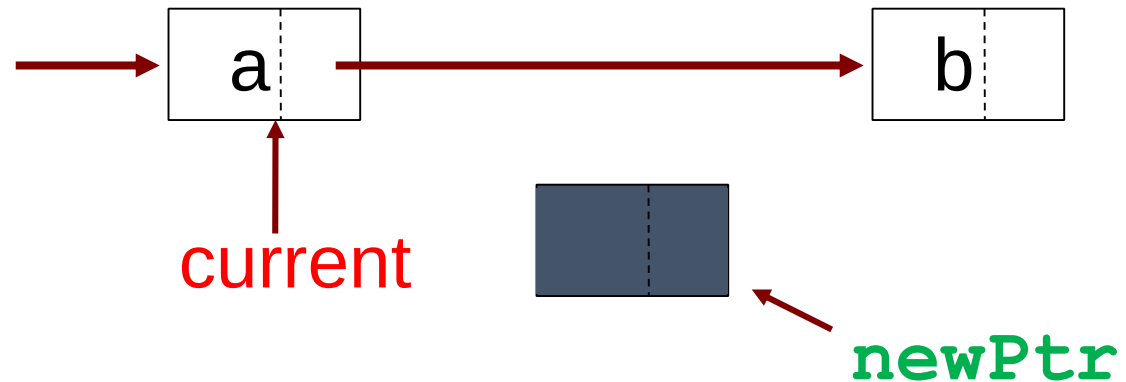
- เพิ่มโหนด X หลังตำแหน่งที่ current ชี้



การเพิ่ม โหนดในลิงค์ลิสต์

- เพิ่มโหนด X หลังตำแหน่งที่ current ชี้

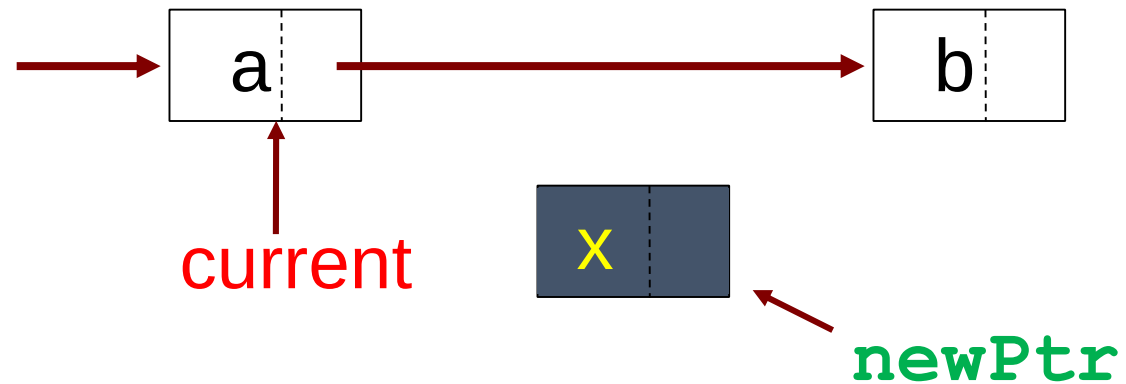
```
newPtr= malloc(sizeof(LISTNODE) )
```



การเพิ่มโหนดในลิงก์ลิสต์ (2)

- เพิ่มโหนด X หลังตำแหน่งที่ $current$ ชี้

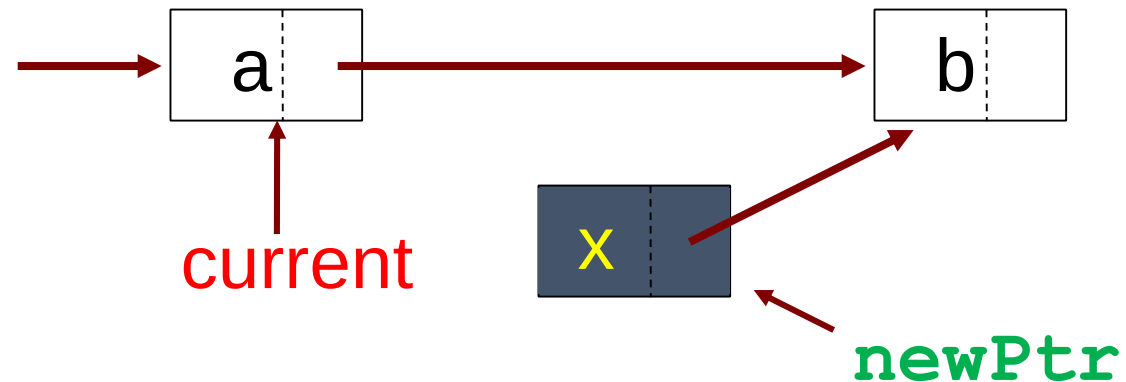
```
newPtr= malloc(sizeof(LISTNODE) )  
newPtr->info = 5
```



การเพิ่มโหนดในลิงก์ลิสต์ (3)

- เพิ่มโหนด X หลังตำแหน่งที่ $current$ ชี้

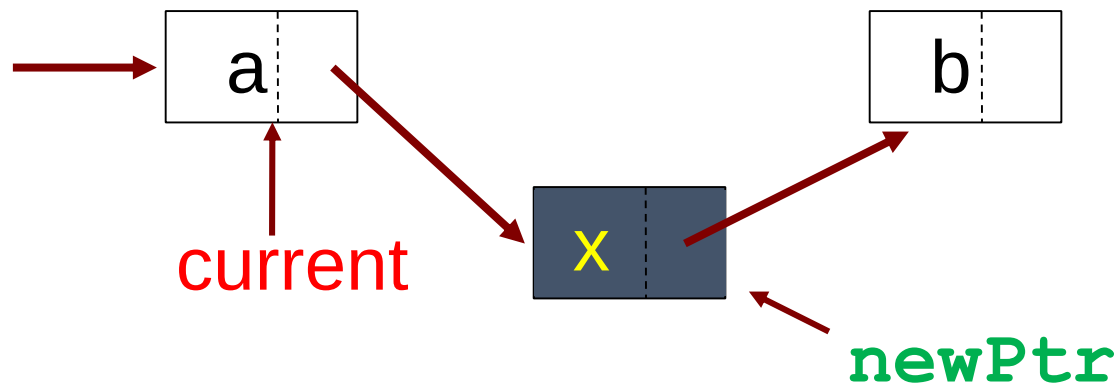
```
newPtr = malloc(sizeof(LISTNODE))  
newPtr->info = 5  
newPtr->next = current->next;
```



การเพิ่มโหนดในลิงก์ลิสต์ (4)

- เพิ่มโหนด X หลังตำแหน่งที่ current ชี้

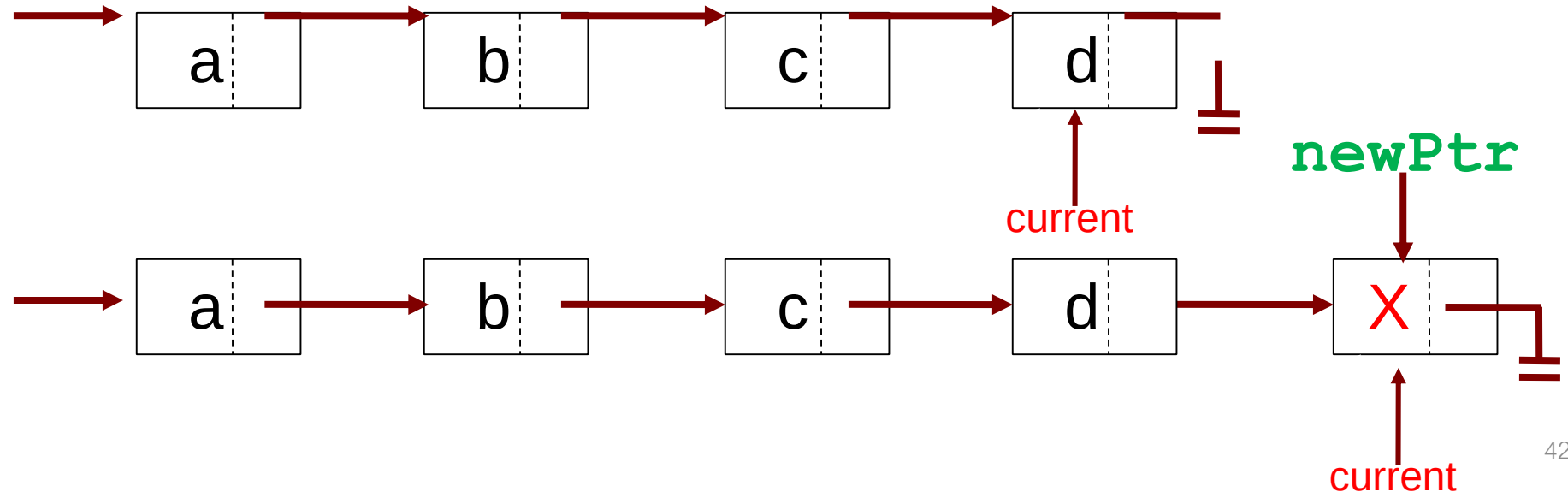
```
newPtr= malloc(sizeof(LISTNODE))
newPtr->info = 5
newPtr->next = current->next;
current->next = newPtr;
```



การเพิ่มโหนดในลิงค์ลิสต์ (Append)

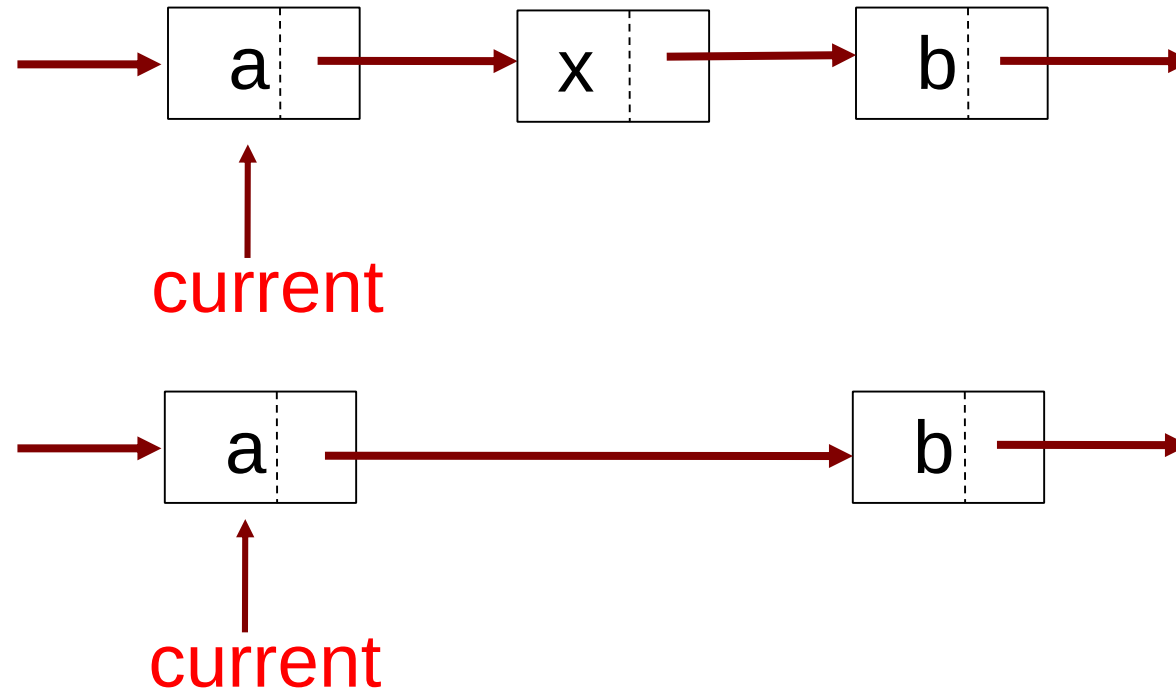
- เพิ่มโหนด X ที่ตำแหน่งสุดท้ายท้ายของลิงค์ลิสต์

```
newPtr= malloc(sizeof(LISTNODE))
newPtr->info = X
current->next = newPtr;
current = newPtr;
current->next = null;
```



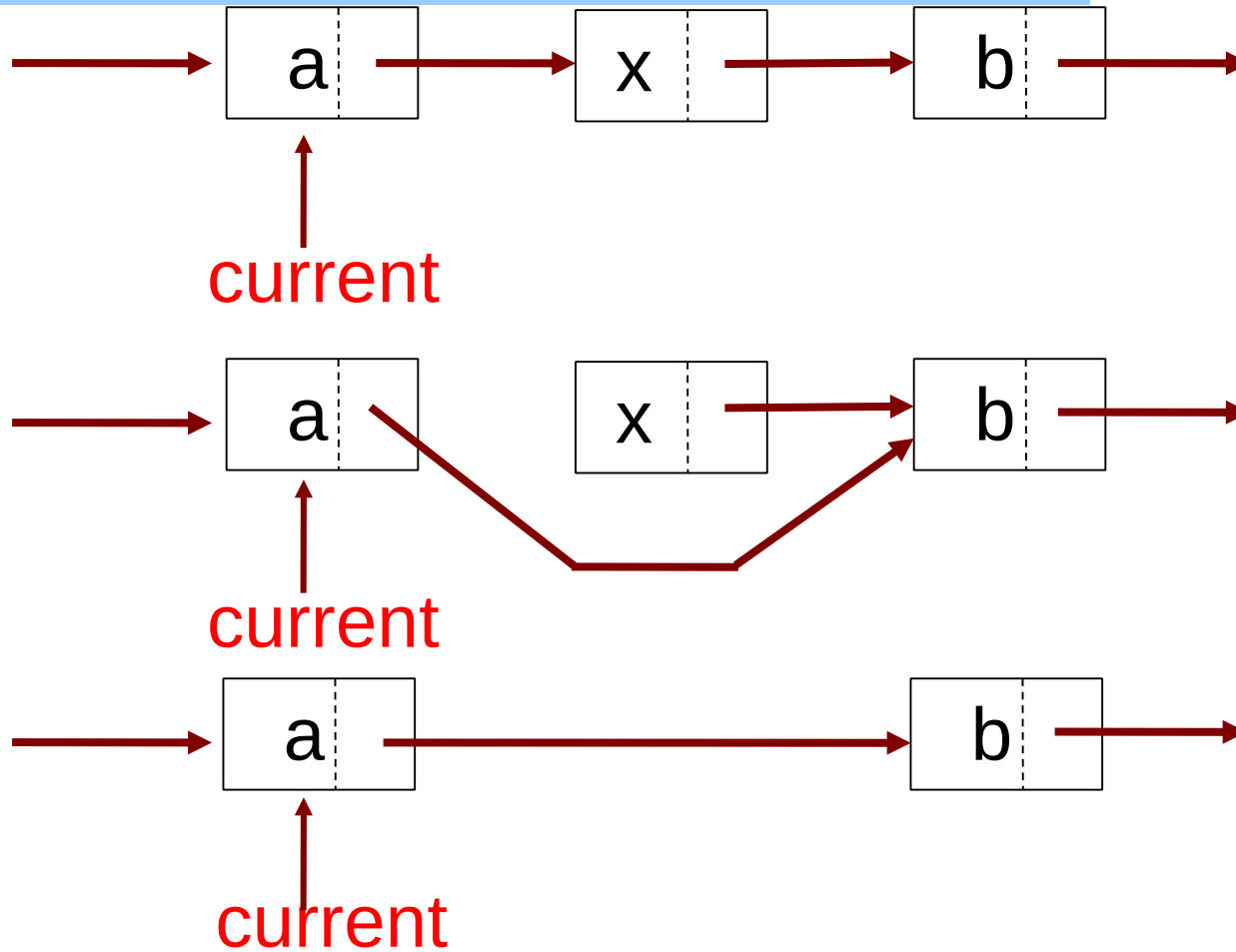
การลบโหนดในลิงก์ลิสต์

- ลบโหนด X หลังตำแหน่งที่ $current$ ชี้



การลบโหนดในลิงค์ลิสต์ (2)

```
current.next = current.next.next;
```



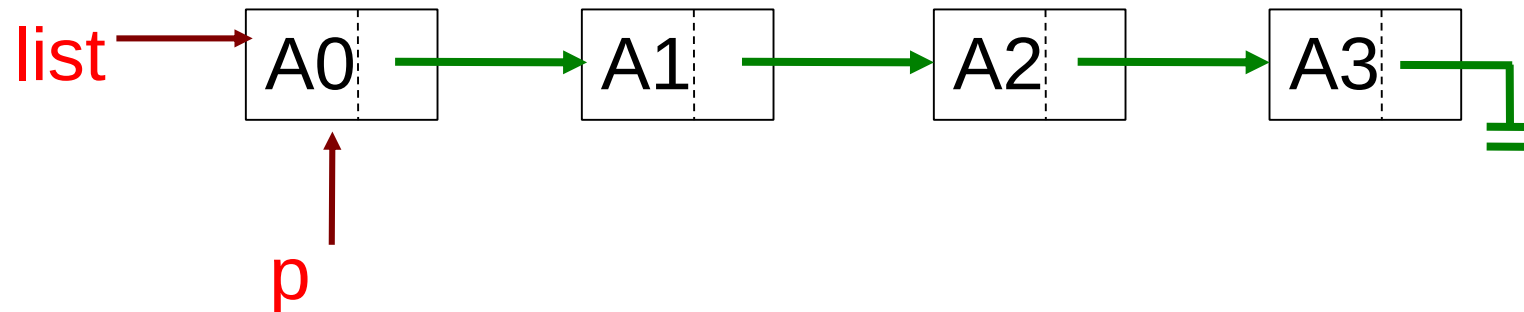
การท่องไปในลิงค์ลิสต์

- ถ้าข้อมูลอยู่ในอาร์เรย์:

```
for (index = 0; index < length; index++)
    printf (a[index]);
```

- ถ้าข้อมูลอยู่ในลิงค์ลิสต์

```
position p
for(p=list; p!=null; p=p.next)
    printf (p->info);
```



Implementing a Linked List

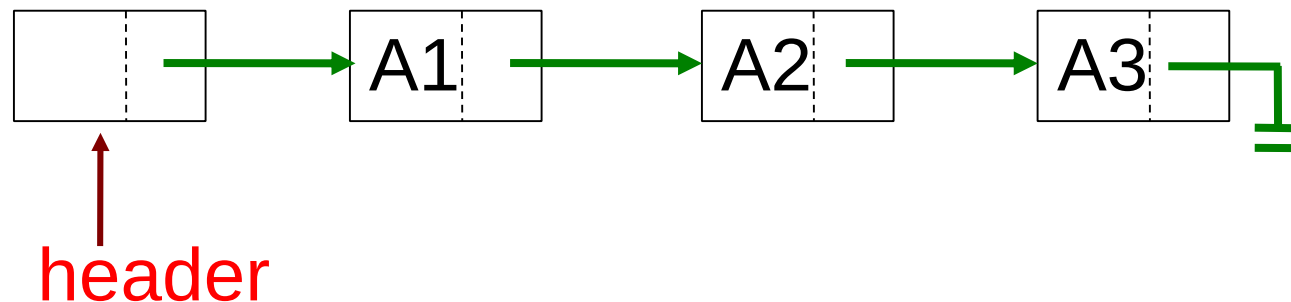
- So what would a Linked List implementation look like?

```
typedef struct node *ptrNode;  
typedef ptrNode listPtr;  
typedef ptrNode position;  
  
struct node{  
    int info;  
    int count;  
    position next;  
};
```

- จะเกิดอะไรขึ้นถ้า
 - ลบโหนดแรก?
 - เพิ่มข้อมูลหน้าโหนดแรก?

Header Nodes

- การใส่สมาชิกลงหัวลิสต์หรือการนำสมาชิกออกจากหัวลิสต์เป็นกรณีพิเศษ เราสามารถให้มีหัวลิสต์ปลอม เรียกว่า header หรือ dummy node ได้ ถ้าทำอย่างนี้ทุกๆ โหนดก็จะมีโหนดนำหน้า ทำให้รายละเอียดการโค้ดเหมือนกันหมด
 - เป็นโหนดที่ไม่มีข้อมูลอยู่ภายใน ทำหน้าที่ชี้ไปที่โหนดจริง ๆ ในลิงค์ลิสต์
 - การไปที่สมาชิกตัวแรกของ linked list กำหนดโดยให้ **current** ไปที่ **header.next;**
 - ลิงค์ลิสต์จะว่าง ถ้า **header.next == null;**
- การค้นหาข้อมูลจะข้าม **header.**

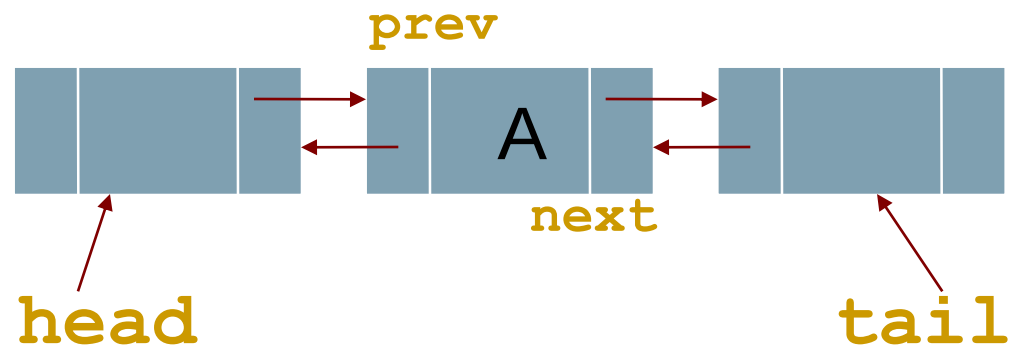


คุณสมบัติของ Linked List

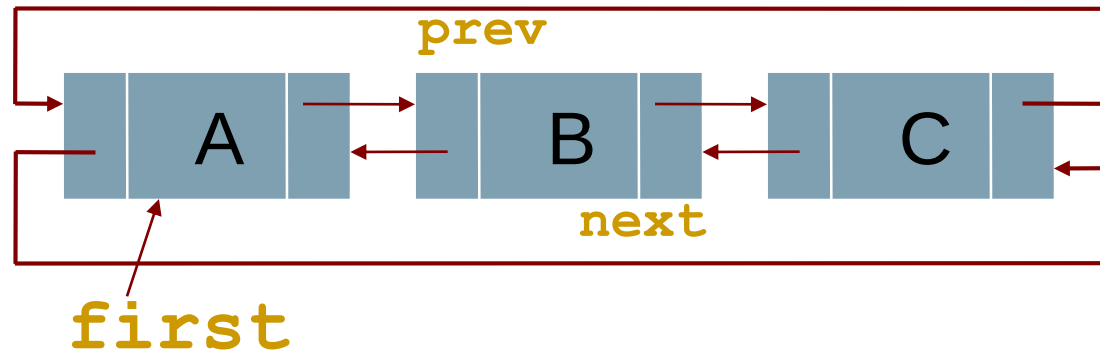
- การวิเคราะห์เวลา Running Time
 - การเพิ่มโหนดต่อจากโหนดก่อนหน้า หรือโหนดสุดท้ายใช้เวลา $O(1)$
 - การลบโหนดต่อจากโหนดก่อนหน้า หรือโหนดแรกใช้เวลา $O(1)$
 - การค้นหาข้อมูลใช้เวลา $O(n)$
 - การดึงข้อมูล ณ ตำแหน่งปัจจุบันใช้เวลา $O(1)$
- ประโยชน์
 - สามารถเพิ่มโหนดได้เรื่อย ๆ
 - ง่ายต่อการอ่าน ลบ เพิ่ม ข้อมูลของโหนดแรกหรือโหนดสุดท้าย
- ข้อเสีย
 - เสียเวลาในการเรียก **malloc**
 - เสียพื้นที่สำหรับเก็บตัวชี้ที่จะชี้ไปที่ตำแหน่งของโหนดต่อไป

ลิงก์ลิสต์ประเภทอื่น ๆ

- Doubly-linked lists:** ทุก ๆ โหนดใน list จะเก็บ pointer ชี้ไปที่โหนดก่อนหน้า และโหนดถัดไป.



- Circular-linked lists:** โหนดสุดท้ายจะชี้ไปที่โหนดแรกโดยจะใช้ Header ด้วยก็ได้



Doubly-linked lists: Wrong InsertNext

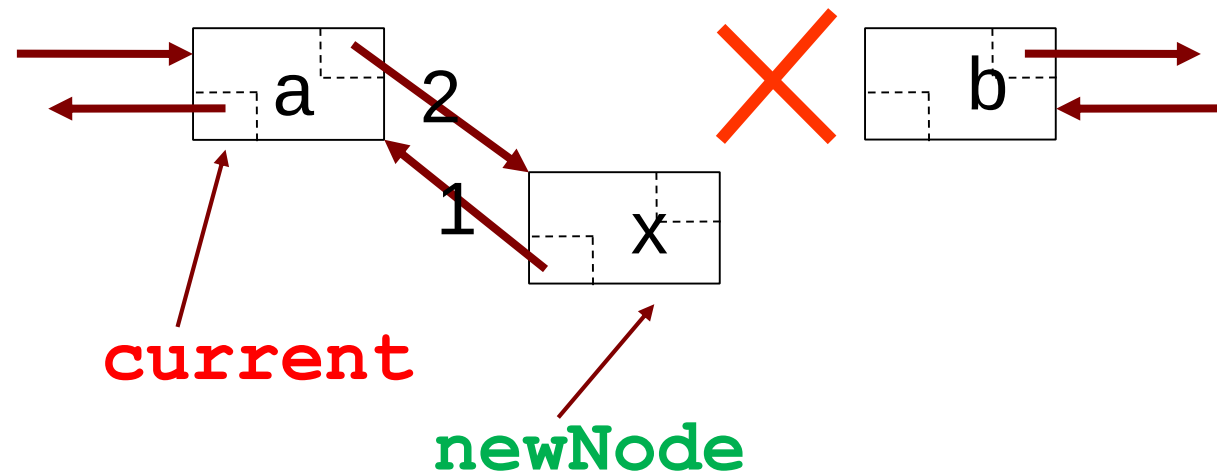
```
newNode = malloc(sizeof(LISTNODE));
```

```
1 newNode.prev = current;
```

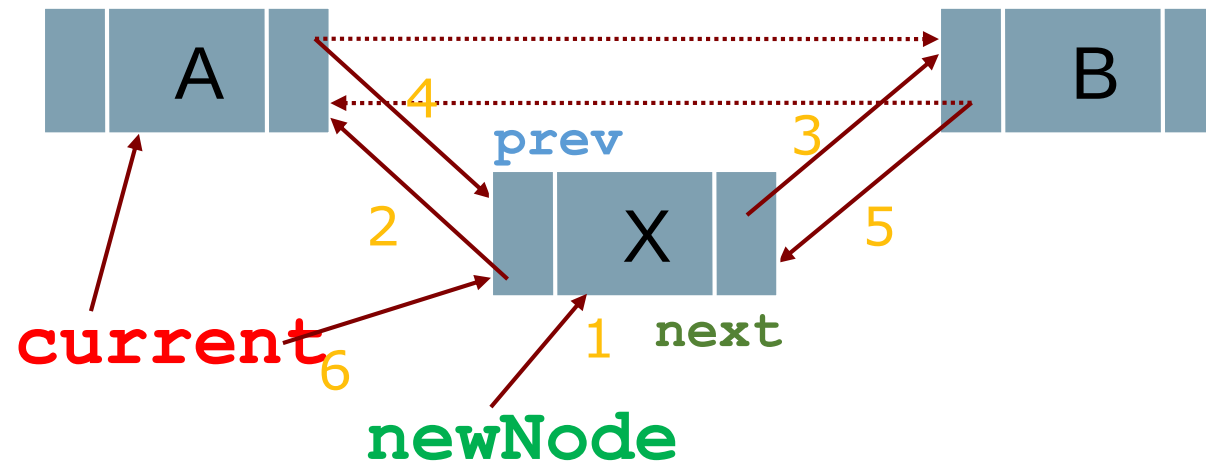
```
2 newNode.prev.next = newNode;
```

...

...



Doubly-linked lists: insertNext



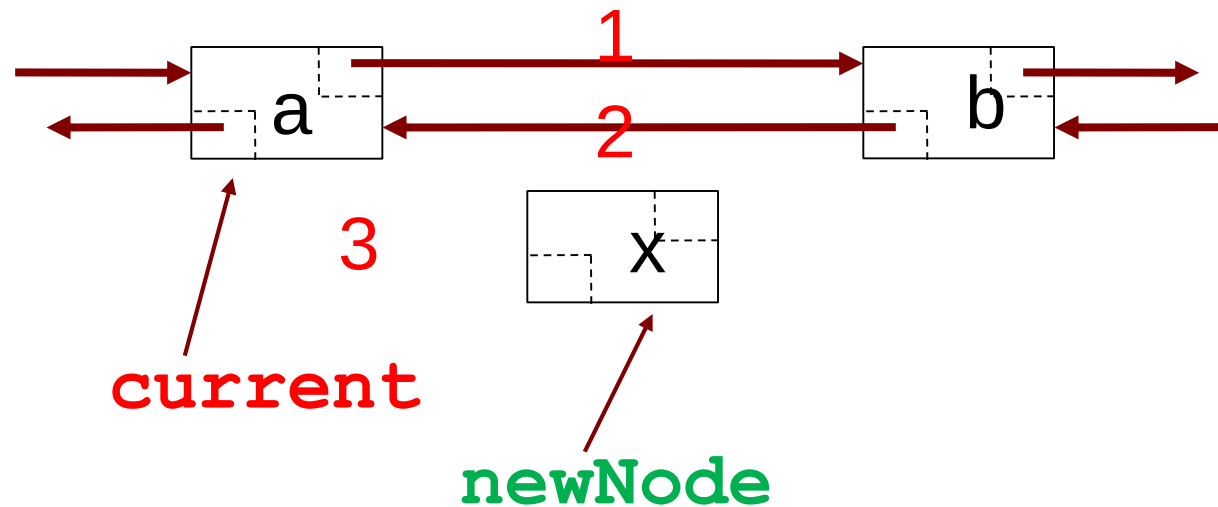
```

1 newNode = malloc(sizeof(LISTNODE));
2 newNode.prev = current;
3 newNode.next = current.next;
4 newNode.prev.next = newNode;
5 newNode.next.prev = newNode;
6 current = newNode;

```

Doubly-linked lists: DeleteCurrent

```
1 current.prev.next = current.next;  
2 current.next.prev = current.prev;  
3 current = current.prev;
```



ตัวอย่าง Linked List ในภาษา C++

```
Class Node{
public:
    int value;
    Node *prev;
    Node *next;

    Node(int value){
        this->value = value;
        this->prev = NULL;
        this->next = NULL;
    }
};
```

```
Class DoublyLinkedList{
public:
    Node *head; Node *tail;
    int size = 0;
    DoublyLinkedList(){
        head = new Node(0); tail = new Node(0);
        head->next = tail; tail->prev = head;
        size = 0;
    }
    PrintList(){
        for(Node *h = head; h->next != NULL; h = h->next)
            cout << h->value << " ";
    }
    PrintListReverse()
        for(Node *t = tail; t->prev != NULL; t = t->prev)
            cout << t->value << " ";
    }
    ...
};
```

ตัวอย่าง Linked List ในภาษา C++ (2)

```
...
InsertAt(int index, int value){
    int my_index = 0;
    for(Node *h = head; h->next != NULL; h = h->next){
        if(my_index == index){
            Node *n = new Node(value);
            n->prev = h; n->next = h->next;
            h->next->prev = n; h->next = n;
        }
        my_index++;
    }
    size++;
}
InsertRear(int value){
    Node *n = new Node(value);
    n->prev = tail->prev; n->next = tail;
    n->prev->next = n; tail->prev = n;
    size++;
}
```

```
...
RemoveAt(int index){
    int my_index = 0;
    for(Node *h = head; h->next != NULL; h = h->next){
        if(my_index == index){
            h->next->prev = NULL;
            h->next = h->next->next;
            h->next->prev = h;
        }
        my_index++;
    }
    size--;
}
RemoveRear(){
    tail->prev->next = NULL;
    tail->prev = tail->prev->prev;
    tail->prev->next = tail;
    size--;
}
```

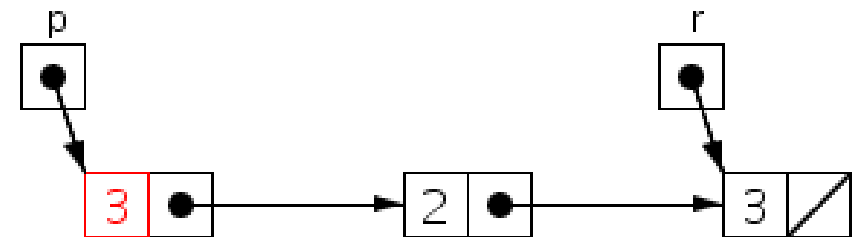
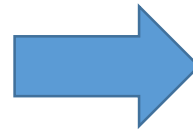
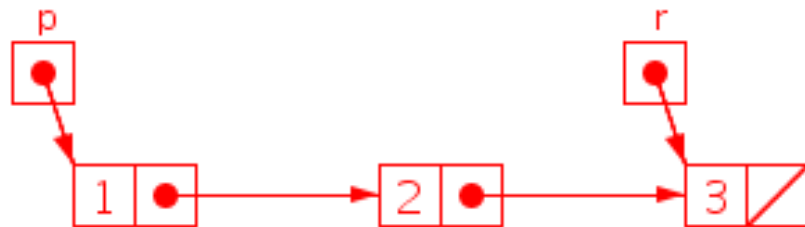
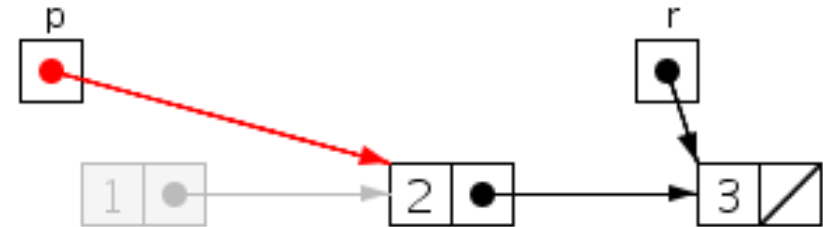
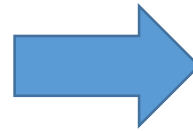
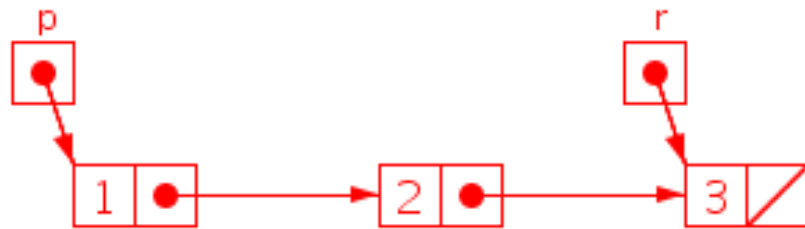
Array versus Linked Lists

- Arrays are suitable for:
 - Inserting/deleting an element at the end.
 - Randomly accessing any element.
 - Searching the list for a particular value.
- Linked lists are suitable for:
 - Inserting an element.
 - Deleting an element.
 - Applications where sequential access is required.
 - In situations where the number of elements cannot be predicted beforehand.

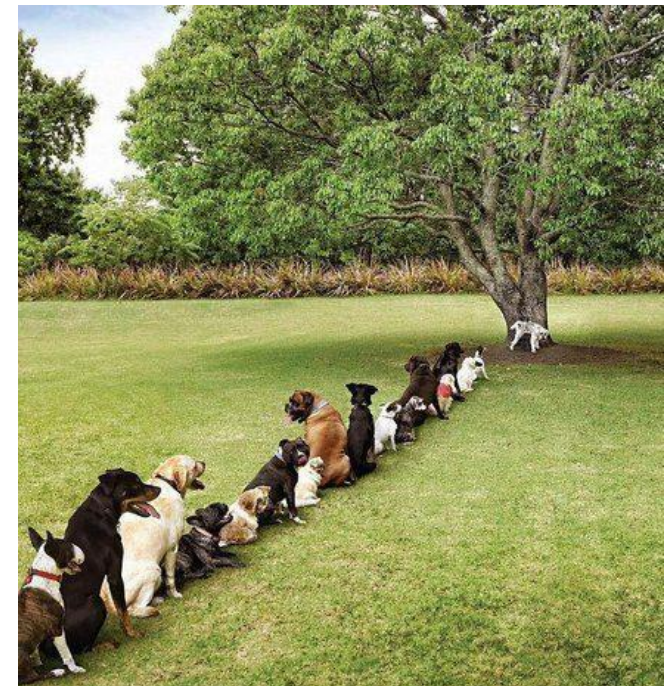
ข้อเปรียบเทียบของประสิทธิภาพของการดำเนินงาน

ฟังก์ชัน	ประสิทธิภาพของ List ที่สร้างด้วย	
	Array	Nodes
create	$O(1)$	$O(1)$
insert at the end	$O(1)$	$O(1)$
insert at given index	$O(n)$	$O(n)^*$
remove by value	$O(n)$	$O(n)^*$
remove by index	$O(n)$	$O(n)^*$
search by value	$O(n)^*$	$O(n)$
get value by index	$O(1)$	$O(n)$
size	$O(1)$	$O(1)$

แบบฝึกหัด



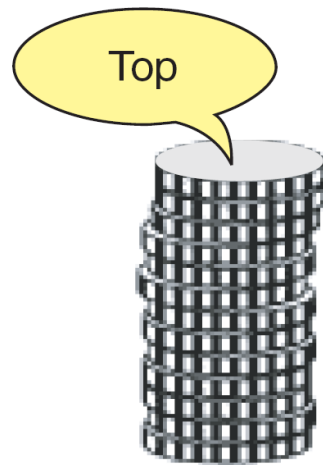
สแตก และ แควคอย (Stacks and Queues)



Outline

- ADT Stacks
 - Basic operations
 - Examples of use
 - Implementations
 - Array-based and linked list-based
- ADT Queues
 - Basic operations
 - Examples of use
 - Implementations
 - Array-based and linked list-based
- Stack and Queue Applications
 - Balanced Symbol Checker
 - Postfix Machines

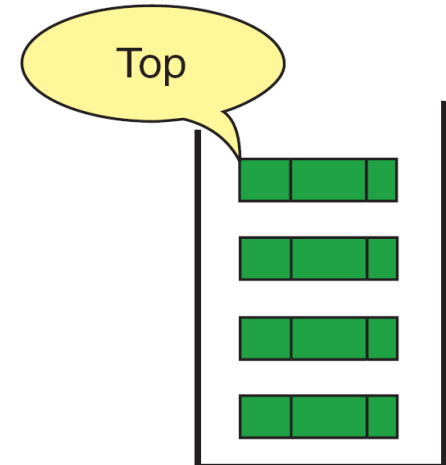
สแตก (Stacks)



Stack of Coins



Stack of Books



Computer Stack

Linear Data Structures

- กลุ่มของข้อมูลที่มีโครงสร้างการจัดเก็บในรูปแบบ 1 มิติ หรือแบบเชิงเส้น **linearly**.
- **Stack**: โครงสร้างข้อมูลแบบเชิงเส้น (a linear data structure) ที่การเข้าถึงข้อมูลทำได้เฉพาะข้อมูลล่าสุดที่ป้อนเข้าไปเท่านั้น (**most recently inserted**)
- **Queue**: โครงสร้างข้อมูลแบบเชิงเส้น (a linear data structure) ที่การเข้าถึงข้อมูลทำได้เฉพาะข้อมูลแรกสุดที่ป้อนเข้าไปเท่านั้น (**least recently inserted**)
- ทั้งสองโครงสร้างข้อมูลสามารถทำได้โดย **list** โดย อาร์เรย์(array) หรือลิงค์ลิสต์ (linked list)

โครงสร้างข้อมูลแบบ Stack

- โครงสร้างข้อมูลแบบ Stack เป็นโครงสร้างข้อมูลที่น่าข้อมูลเข้าหรือออกได้ **ทางเดียวเท่านั้น**
- โครงสร้างข้อมูลแบบ Stack เป็นแบบ **เข้าทีหลังออกก่อน** (*Last in First Out-LIFO*)
- ข้อมูลล่าสุดจะถูกเพิ่ม **push** เข้าไปใน stack.
- ข้อมูลล่าสุดที่ถูกเพิ่มจะถูกนำออกมาใช้ **pop** จาก stack.
- ข้อมูลล่าสุดที่ถูกเพิ่มคือ **top** ของ stack.
- ฟังก์ชันต่าง ๆ เหล่านี้จะทำในรูปแบบ **constant time: $O(1)$.**



โครงสร้างข้อมูลแบบ Stack (2)

- โครงสร้างข้อมูลแบบ Stack มีฟังก์ชันที่สามารถจัดการ Stack ได้ดังนี้
 - การสร้าง Stack (Create)
 - การใส่ข้อมูลใน Stack (Push)
 - การนำข้อมูลออกจาก Stack (pop)
 - การทดสอบว่า Stack ว่างหรือไม่ (isEmpty)
 - การทดสอบว่า Stack เต็มหรือไม่ (isFull)
 - การหาจำนวนสมาชิกที่อยู่ใน Stack (size)



การสร้างสแตกด้วยแถวลำดับ

- **การสร้าง Stack ด้วยแถวลำดับ (Array)** หมายถึง การเลือกการแทนที่ข้อมูลของ stack ซึ่งเป็นการจัดสรรพื้นที่หน่วยความจำแบบ Static คือ มีการกำหนดขนาดของ stack ล่วงหน้าว่าจะมีขนาดเท่าไรและมีการจัดสรรหน่วยความจำให้เลย

การสร้างสแตกด้วยแถวลำดับ (2)

```
#define MAXSIZE 10;  
int  stackArray[MAXSIZE];  
int  StackPointer=-1
```

- การนำสมาชิกที่ส่งมาเข้าไปยังบนสุดของ Stack

```
void push(int y)  
{  
    if(StackPointer >MAXSIZE)  
    {  
        printf("          STACK FULL");  
        return;  
    }  
    else  
    {  
        StackPointer ++;  
        stackArray [StackPointer]=y;  
    }  
}
```

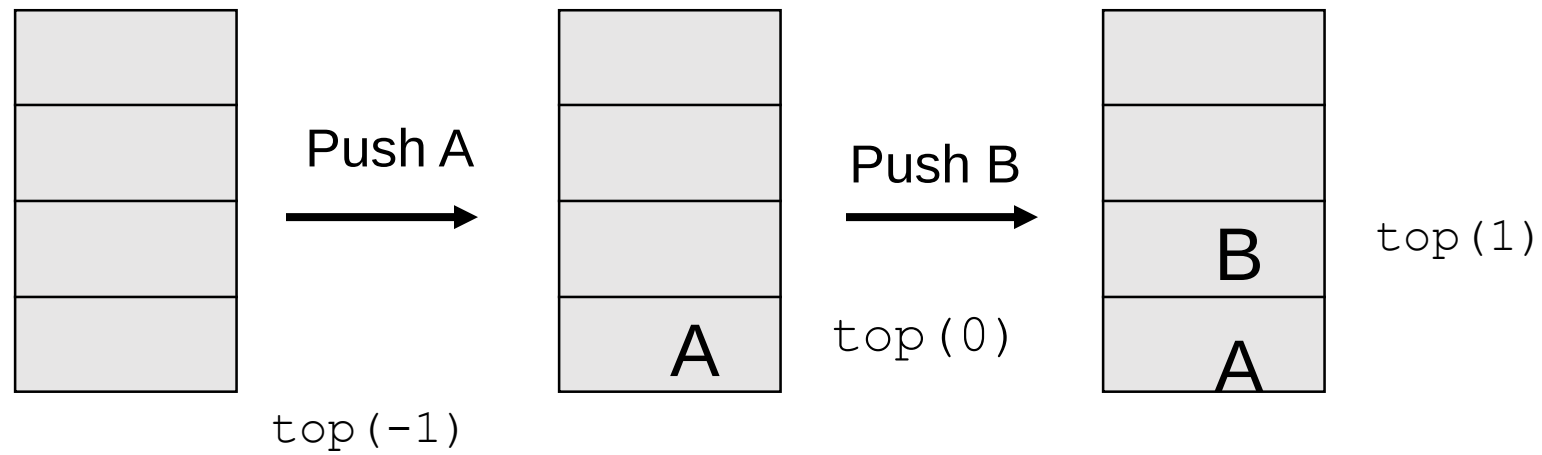
การสร้างสแตกด้วยแถวลำดับ (3)

```
#define MAXSIZE 10;  
int  stackArray[MAXSIZE];  
int  StackPointer=-1
```

- การนำค่าที่อยู่บนสุดของ stack ออกและส่งคืนค่านั้น

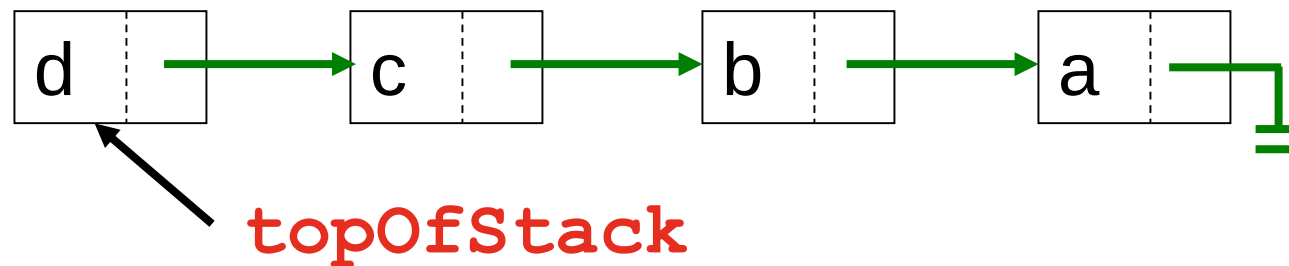
```
int pop()  
{  
    int a;  
    if(stackPointer <=0){  
        printf("STACK EMPTY");  
        return 0;  
    }  
    else{  
        a=stackArray[stackPointer];  
        stackPointer --;  
    }  
    return(a);  
}
```

ตัวอย่าง



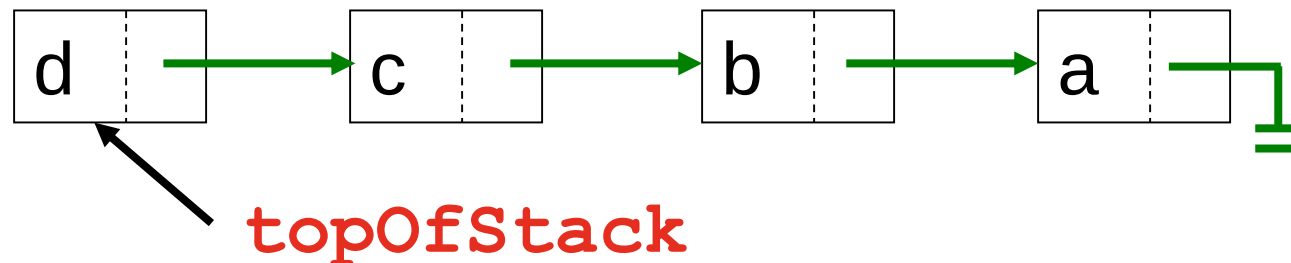
การสร้างสแตกด้วยลิงค์ลิสต์

- การสร้าง Stack ด้วยลิงค์ลิสต์ (Linked List) หมายถึง การเลือกแทนที่ข้อมูลของ Stack ด้วย Linked List ซึ่งเป็นการจัดสรรพื้นที่ของหน่วยความจำแบบพลวัต (dynamic) คือ หน่วยความจำจะถูกจัดสรรเมื่อมีการขอใช้จริงระหว่างการประมวลผลโปรแกรมผ่านตัวแปรชนิดพอยน์เตอร์ pointer
- การเลือกสร้าง Stack ด้วย Linked List สแตกจะไม่เต็มในขณะที่ยังสามารถจัดสรรเนื้อที่ในหน่วยความจำได้



การสร้างสแตกด้วยลิงค์ลิสต์ (2)

- สมาชิกตัวแรกในลิงค์ลิสต์จะเป็น= top of stack (ถ้า Stack ว่าง: null)
- push (Thing x) :
 - สร้างโหนดที่มี x อยู่ข้างใน
 - เอาโหนดดังกล่าวใส่เข้าไปหน้าลิงค์ลิสต์
- pop () :
 - ลบโหนดแรกของลิงค์ลิสต์ ที่ตำแหน่ง top of stack
- top () :
 - คืนข้อมูลของโหนดแรกที่อยู่บน top of stack

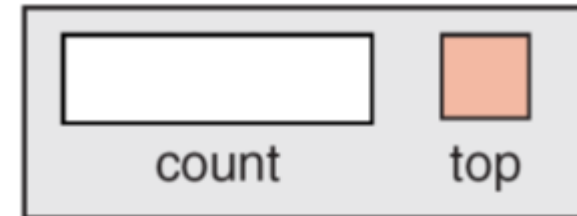


การสร้างสแตกด้วยลิงค์ลิสต์ (3)

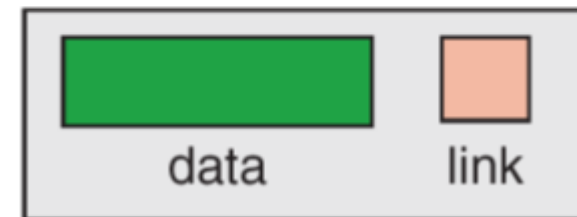
```
//Global Declarations
typedef struct node
{
    int          data;
    struct node* link;
} STACK_NODE;
```

```
typedef struct
{
    int          count;
    STACK_NODE* top;
} STACK;
```

```
//Function Declarations
void insertData (STACK* pStack);
void print      (STACK* pStack);
bool push       (STACK* pList, int
dataIn);
bool pop        (STACK* pList, int*
dataOut);
```



Stack Head Structure



Stack Node Structure

การสร้างสแตกด้วยลิงค์ลิสต์ (4)

```
bool push (STACK* pStack, int dataIn)
{
    //Local Declarations
    STACK_NODE* pNew;
    bool success;
    pNew = (STACK_NODE*)malloc(sizeof (STACK_NODE));
    if (!pNew)
        success = false;
    else
    {
        pNew->data = dataIn;
        pNew->link = pStack->top;
        pStack->top = pNew;
        pStack->count++;
        success = true;
    } // else
    return success;
} // push
```

การสร้างสแตกด้วยลิงค์ลิสต์ (5)

```
bool pop (STACK* pStack, int* dataOut)
{
    //Local Declarations
        STACK_NODE* pDlt;
        bool          success;

    //Statements
        if (pStack->top)                // Test for Empty Stack
        {
            success = true;
            *dataOut = pStack->top->data;
            pDlt     = pStack->top;
            pStack->top = (pStack->top)->link;
            pStack->count--;
            free (pDlt);
        } // else
        else
            success = false;
        return success;
} // pop
```

การสร้างสแตกด้วยลิงค์ลิสต์ (6)

```
void insertData (STACK* pStack)
{
    //      Local Declarations
    int  numIn;
    bool success;

    //      Statements
    printf("Creating numbers: ");
    for (int nodeCount = 0; nodeCount < 10; nodeCount++)
    {
        // Generate random number
        numIn = rand() % 999;
        printf("%4d", numIn);
        success = push(pStack, numIn);
        if (!success)
        {
            printf("Error 101: Out of Memory\n");
            exit (101);
        } // if
    } // for
    printf("\n");
    return;
} // insertData
```

การสร้างสแตกด้วยลิงค์ลิสต์ (7)

```
void print (STACK* pStack)
{
    //      Local Declarations
    int printData;

    //      Statements
    printf("Stack contained: ");
    while (pop(pStack, &printData))
        printf("%4d", printData);
    return;
} // print
```

การสร้างสแตกด้วยลิงค์ลิสต์ (8)

```
int main (void)
{
    //      Local Declarations
            STACK* pStack;

    //      Statements
    printf("Beginning Simple Stack Program\n");

    pStack = malloc(sizeof(STACK));
    if (!pStack)
        printf("Error allocating stack"), exit(100);

    pStack->top    = NULL;
    pStack->count = 0;
    insertData (pStack);
    print      (pStack);

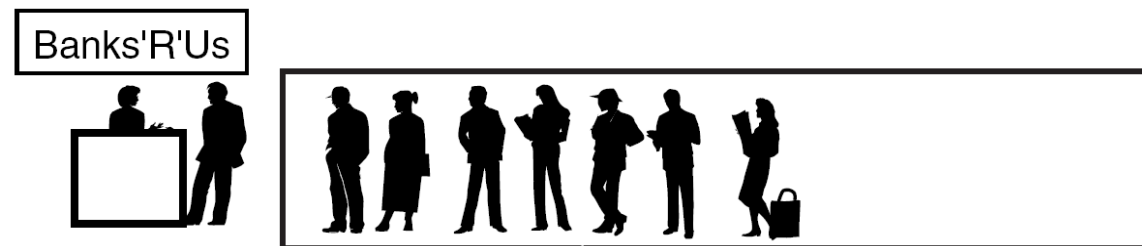
    printf("\nEnd Simple Stack Program\n");
    return 0;
} // main
```

ข้อเปรียบเทียบของประสิทธิภาพของการดำเนินงาน

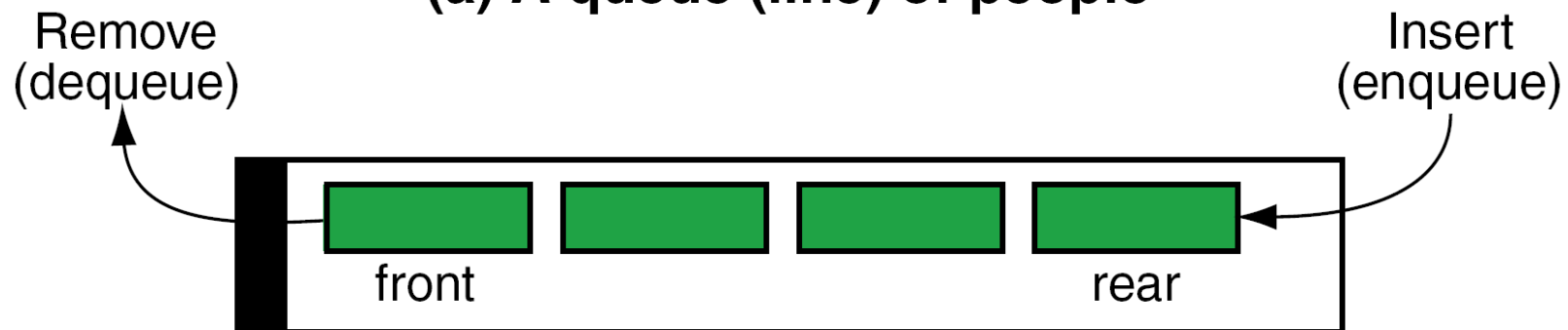
ฟังก์ชัน	ประสิทธิภาพของ stack ที่ สร้างด้วย Array	ประสิทธิภาพของ stack ที่ สร้างด้วย Linked list
create	$O(1)$	$O(1)$
push	$O(1)$	$O(1)$
pop	$O(1)$	$O(1)$
isFull	$O(1)$	-
isEmpty	$O(1)$	$O(1)$
size	$O(1)$	$O(1)$

แถวคอย (Queues)

- แถวคอย หรือ คิว



(a) A queue (line) of people



(b) A computer queue

โครงสร้างข้อมูลแบบคิว



- คิวมีโครงสร้างข้อมูลแบบเชิงเส้น (linear)
- โครงสร้างข้อมูลแบบคิวเป็นแบบ *เข้าก่อนออกก่อน (First in First Out-FIFO)*

โครงสร้างข้อมูลแบบคิว (2)



- โครงสร้างข้อมูลแบบคิว มีฟังก์ชันที่สามารถจัดการ Queue ได้ดังนี้
 - การสร้างคิว (Create)
 - การใส่ข้อมูลในคิว (enqueue)
 - การนำข้อมูลออกจากคิว (dequeue)
 - การทดสอบว่าคิวว่างหรือไม่ (isEmpty)
 - การทดสอบว่าคิวเต็มหรือไม่ (isFull)
 - การหาจำนวนสมาชิกที่อยู่ในคิว (size)

การสร้างคิวด้วยแถวลำดับ

- การสร้างคิวด้วยแถวลำดับ (Array) หมายถึง การเลือกการแทนที่ข้อมูลของคิวซึ่งเป็นการจัดสรรพื้นที่หน่วยความจำแบบ Static คือ มีการกำหนดขนาดของคิวล่วงหน้าว่าจะมีขนาดเท่าไรและมีการจัดสรรหน่วยความจำให้เลย

การสร้างคิวด้วยแถวลำดับ (2)

```
#define MAXSIZE 10;
int queue [MAXSIZE];
int front=0;
int rear=0
```

■ การนำสมาชิกป้อนเข้าไปใน Queue

```
void enqueue(int queue[], int *rear, int front, int value)
{
    *rear= (*rear +1) % MAX;
    if(*rear == front)
    {
        printf("The queue is full\n");
        exit(0);
    }
    queue[*rear] = value;
}
```

การสร้างคิวด้วยแถวลำดับ (3)

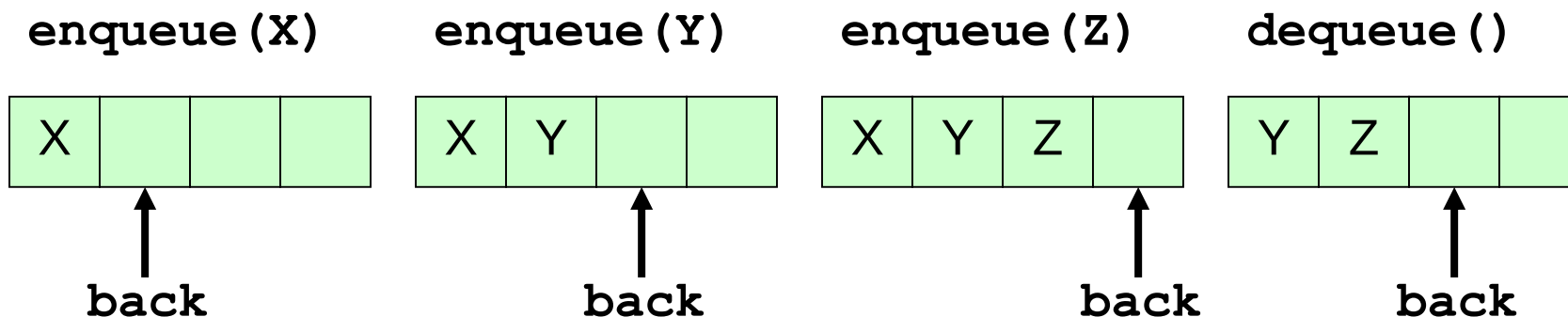
```
#define MAXSIZE 10;  
int queue [MAXSIZE];  
int front=0;  
int rear=0
```

■ การนำสมาชิกออกจาก Queue

```
void deQueue(int queue[], int *front, int rear, int * value)  
{  
    if(*front == rear)  
    {  
        printf("The queue is empty\n");  
        exit(0);  
    }  
    *front = (*front + 1) % MAX;  
    *value = queue[*front];  
}
```

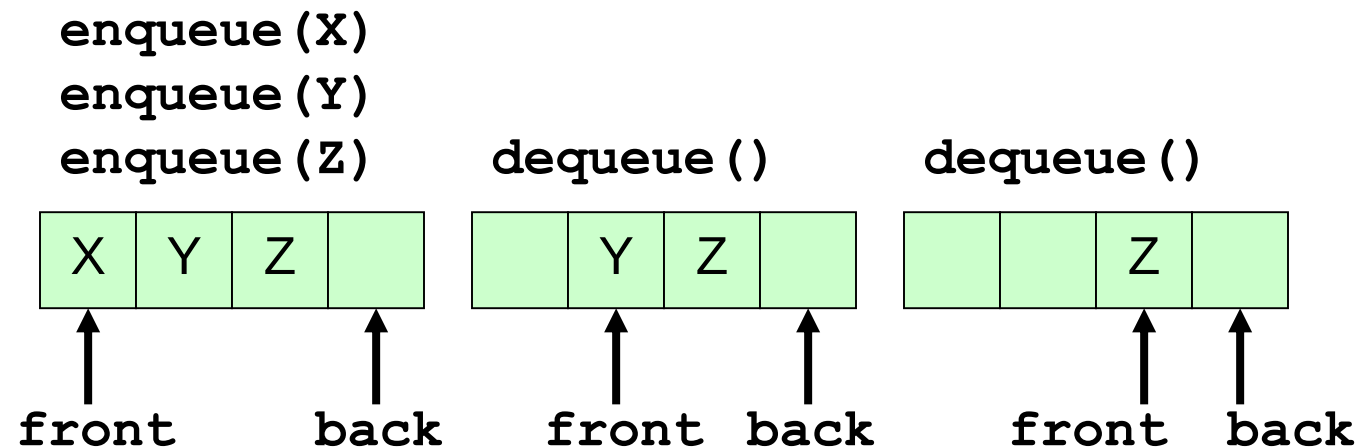
การสร้างคิวด้วยแถวลำดับ: Simple Idea

- เก็บข้อมูลในอาร์เรย์ A
- จัดการกับ index: $back$
- หน้า $queue = A[0]$
- หลัง $queue = A[back]$
- การ Enqueue คือการเก็บข้อมูล ณ $A[back]$ แล้วเพิ่ม index ของ $back++$
- การ Dequeue คือการเอาสมาชิกตัวแรกออกไปแล้วเลื่อนสมาชิกที่เหลือมาข้างหน้า



การสร้างคิวด้วยแถวลำดับ: Better Idea

- สร้าง index: **front** เก็บหัว queue ซึ่งเป็นตำแหน่งหน้าสุดของคิว
- การ Dequeue ทำได้โดยการเพิ่ม index ของ **front**
- การทำงานของ enqueue และ dequeue จะใช้เวลาเป็นค่าคงที่ $O(1)$.
- จะเกิดอะไรขึ้นถ้า enqueue และ dequeue ที่ตำแหน่งสุดท้ายของอาร์เรย์ (length-1)?

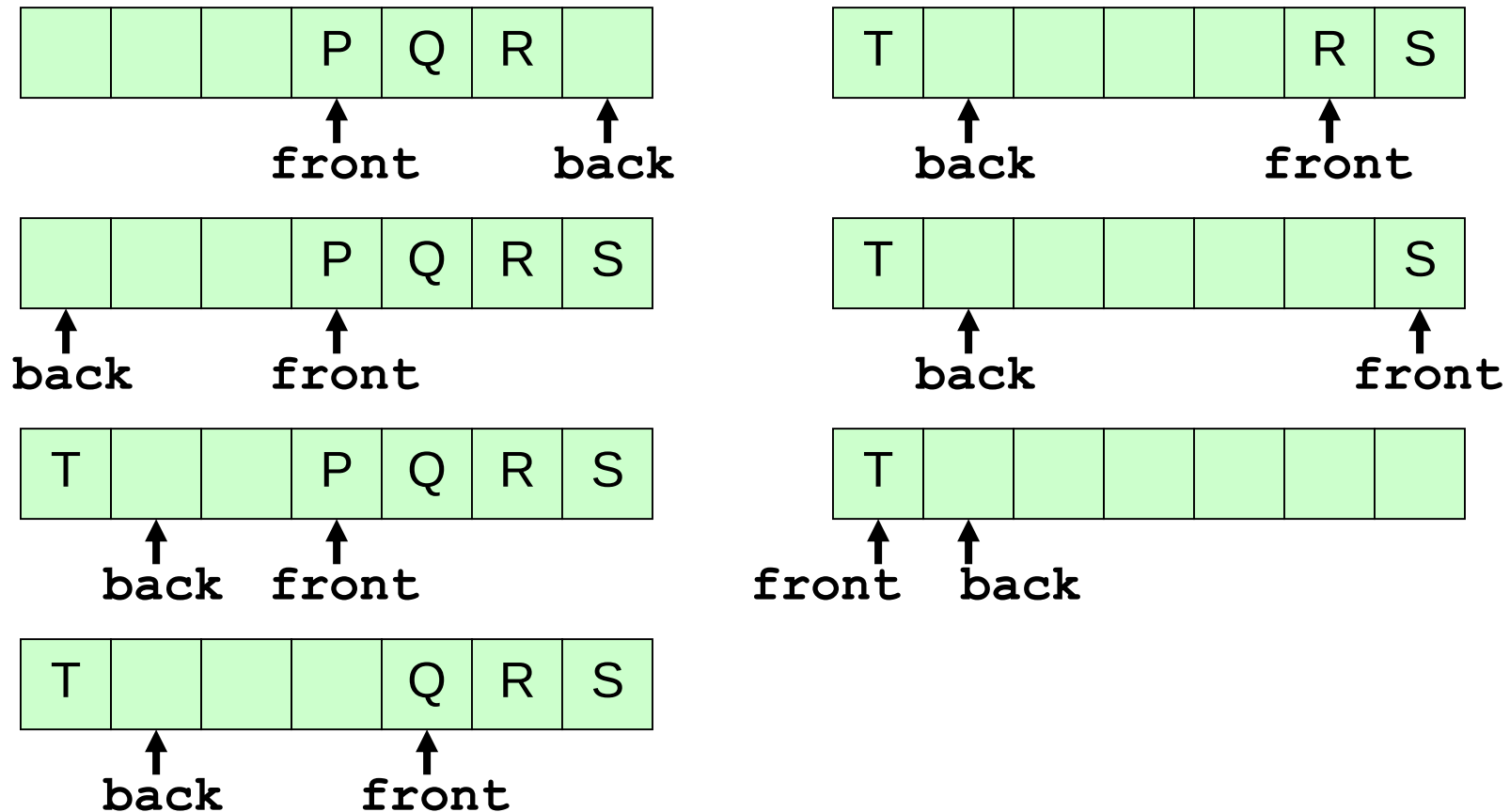


การสร้างคิวด้วยแถวลำดับ: “Circular” Array

- เมื่ออาร์เรย์ตัวที่ `length-1` ถูก enqueue ตัวคิวจะบอกว่าคิวเต็มทั้งที่จริง ๆ ไม่เต็ม.
- วิธีการแก้ปัญหาคือ: `wraparound`
- กลับไปใช้ cell ที่ตำแหน่งเริ่มต้นของอาร์เรย์ที่ยังว่างอยู่ที่เกิดจากการ `dequeue`.
- เมื่อ `front` หรือ `back` เพิ่มและอยู่นอกอาร์เรย์ “*outside array*” ($\geq \text{length}$), ให้เซ็ทเป็น 0.

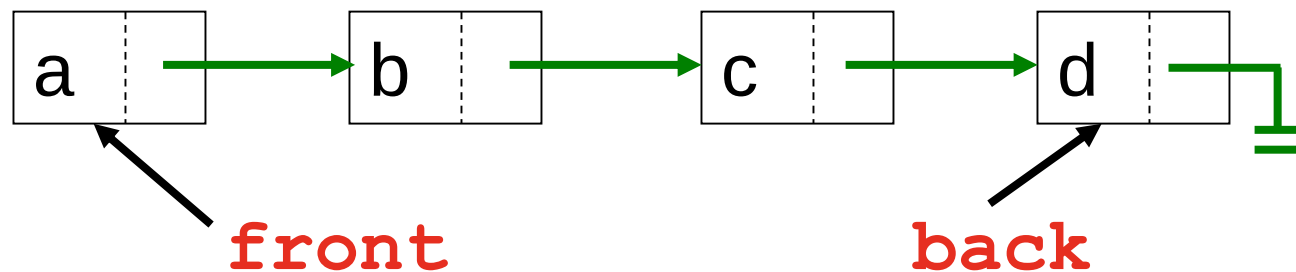
ตัวอย่างของ Circular

- Both **front** and **back** indexes “wraparound” the array. Think of the array as a circle...



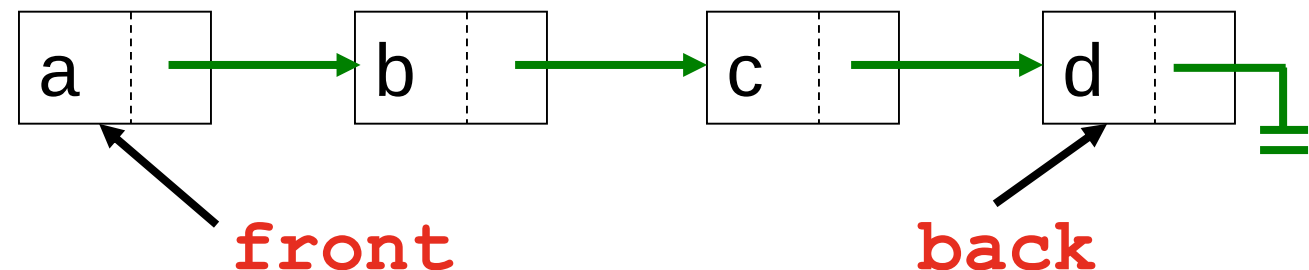
การสร้างคิวด้วยลิงค์ลิสต์

- การสร้างคิวด้วยลิงค์ลิสต์ (Linked List) หมายถึง การเลือกแทนที่ข้อมูลของคิวด้วย Linked List ซึ่งเป็นการจัดสรรพื้นที่ของหน่วยความจำแบบพลวัต (dynamic) คือ หน่วยความจำจะถูกจัดสรรเมื่อมีการขอใช้จริงระหว่างการประมวลผลโปรแกรมผ่านตัวแปรชนิดพอยน์เตอร์ pointer
- การเลือกสร้าง คิวด้วย Linked List คิวจะไม่เต็มในขณะที่ยังสามารถจัดสรรเนื้อที่ในหน่วยความจำได้

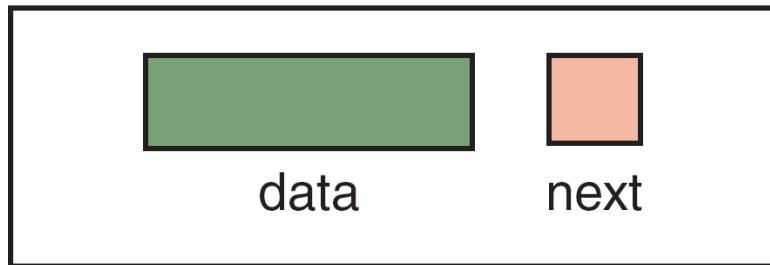


การสร้างคิวด้วยลิงค์ลิสต์ (2)

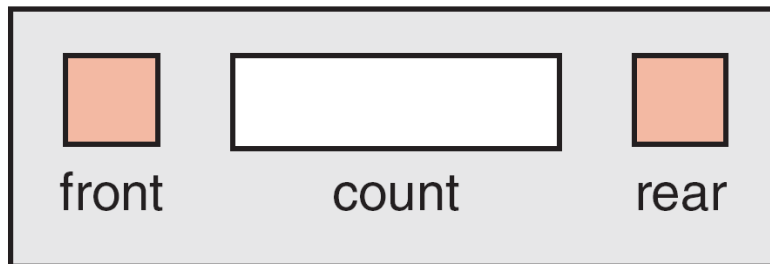
- จัดการกับโหนด 2 โหนดคือ: front & back
- คิวจะว่างเมื่อ: front = back = null.
- enqueue(Thing X):
 - สร้าง node N ที่มี X
 - ถ้าคิวว่าง: front = back = N
 - กรณีอื่นให้ต่อ N เข้าไปในลิงค์ลิสต์ และเปลี่ยน back ใหม่
- dequeue():
 - ลบข้อมูลที่โหนดแรก (กำหนดโดย front)
- getFront():
 - ดึงข้อมูลที่อยู่โหนดแรกออกไป



การสร้างคิวด้วยลิงก์ลิสต์ (3)



Node Structure



Head Structure

```
typedef struct node
{
    int data;
    struct node* next;
} QUEUE_NODE;
```

```
typedef struct
{
    QUEUE_NODE* front;
    int count;
    QUEUE_NODE* rear;
} QUEUE;
```

การสร้างคิวด้วยลิงค์ลิสต์ (4)

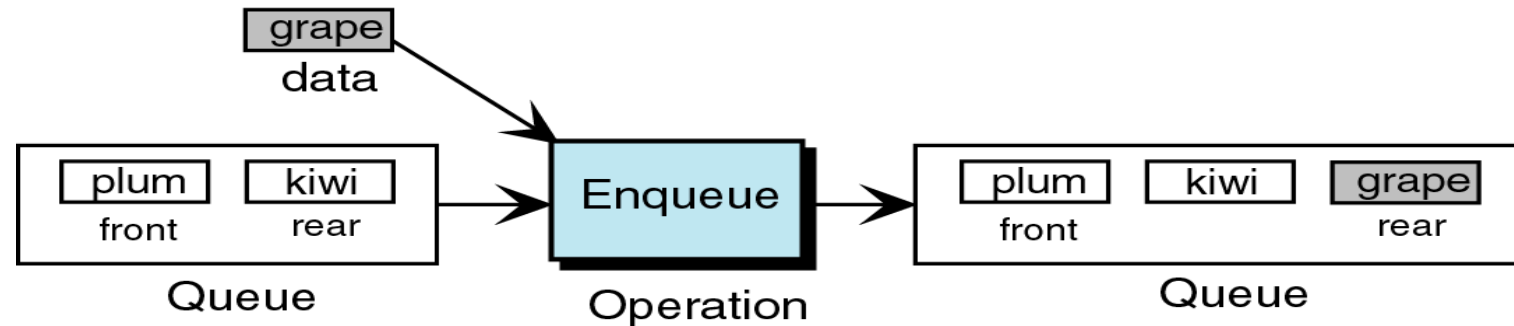
```
bool enqueue (QUEUE* queue, int dataIn)
```

```
{
//      Local Declarations
QUEUE_NODE* newPtr;
```

```
//      Statements
if (!(newPtr =
malloc(sizeof(QUEUE_NODE))))
    return false;
```

```
newPtr->data = dataIn;
newPtr->next = NULL;
```

```
if (queue->count == 0)
    // Inserting into null queue
    queue->front = newPtr;
else
    queue->rear->next = newPtr;
    (queue->count)++;
    queue->rear = newPtr;
    return true;
// enqueue
}
```



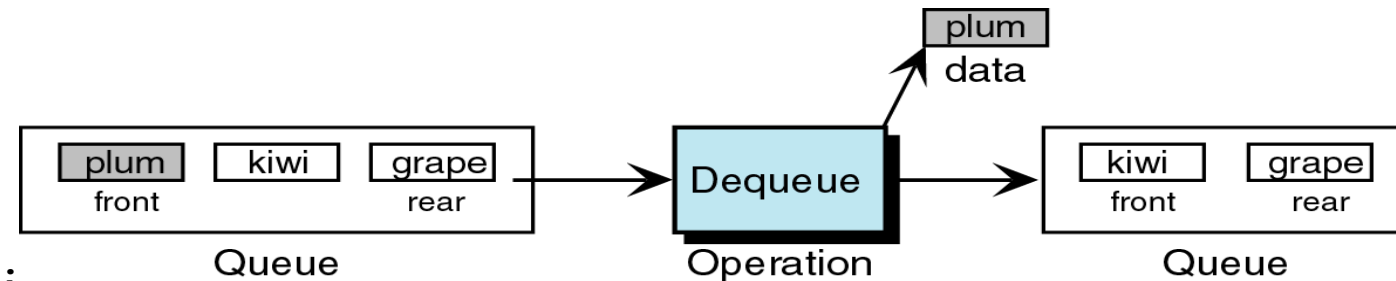
การสร้างคิวด้วยลิงค์ลิสต์ (5)

```
bool dequeue (QUEUE* queue, int* dataOut)
{
    //      Local Declarations
    QUEUE_NODE* deleteLoc;

    //      Statements
    if (!queue->count)
        return false;

    *dataOut = queue->front->data;
    deleteLoc = queue->front;
    if (queue->count == 1)
        // Deleting only item in queue
        queue->rear = queue->front = NULL;
    else
        queue->front = queue->front->next;
    (queue->count)--;
    free (deleteLoc);

    return true;
}
// dequeue
```



การสร้างคิวด้วยลิงก์ลิสต์ (6)

```
typedef struct node
{
    int          data;
    struct node* next;
} QUEUE_NODE;

typedef struct
{
    QUEUE_NODE* front;
    int         count;
    QUEUE_NODE* rear;
} QUEUE;

//      Function Declarations
void insertData (QUEUE* pQueue);
void print      (QUEUE* pQueue);
bool enqueue    (QUEUE* pList, int  dataIn);
bool dequeue    (QUEUE* pList, int* dataOut);
```

การสร้างคิวด้วยลิงค์ลิสต์ (7)

```
void print (QUEUE* pQueue)
{
    //      Local Declarations
    int printData;

    //      Statements
    printf("Queue contained:  ");
    while (dequeue(pQueue, &printData))
        printf("%4d", printData);
    return;
}    // print
```

การสร้างคิวด้วยลิงค์ลิสต์ (8)

```
int main (void)
{
    //      Local Declarations
                QUEUE* pQueue;

    //      Statements
    printf("Beginning Simple Queue Program\n");

    pQueue = malloc(sizeof(QUEUE));
    if (!pQueue)
        printf("Error allocating queue"), exit(100);

    pQueue->front = NULL;
    pQueue->count = 0;
    pQueue->rear  = NULL;

    insertData (pQueue);
    print      (pQueue);

    printf("\nEnd Simple Queue Program\n");
    return 0;
} // main
```

การสร้างคิวด้วยลิงค์ลิสต์ (9)

```
void insertData (QUEUE* pQueue)
{
    //      Local Declarations
    int  numIn;
    bool  success;

    //      Statements
    printf("Creating numbers: ");
    for (int nodeCount = 0; nodeCount < 5; nodeCount++)
    {
        // Generate random number
        numIn = rand() % 999;
        printf("%4d", numIn);
        success = enqueue(pQueue, numIn);
        if (!success)
        {
            printf("Error 101: Out of Memory\n");
            exit (101);
        } // if
    } // for
    printf("\n");
    return;
} // insertData
```

ข้อเปรียบเทียบของประสิทธิภาพของการดำเนินงาน

ฟังก์ชัน	ประสิทธิภาพของคิวที่สร้าง ด้วย Array	ประสิทธิภาพของคิวที่สร้าง ด้วย Linked list
create	$O(1)$	$O(1)$
Enqueue	$O(1)$	$O(1)$
dequeue	$O(1)$	$O(1)$
isFull	$O(1)$	-
isEmpty	$O(1)$	$O(1)$
size	$O(1)$	$O(1)$

ตัวอย่างการประยุกต์ใช้งาน



Stack Application: Balanced Symbols

- โปรแกรมส่วนมากใช้คู่ของสัญลักษณ์ทำการกำหนด blocks
 - (และ)
 - { และ }
 - [และ]
- สัญลักษณ์เปิดต้องคู่กับสัญลักษณ์ปิด
 - { [()] } **OK**
 - { ([)] } **NOT OK**
- สัญลักษณ์ปิดต้องจับคู่กับตัวที่เพิ่งใช้สัญลักษณ์เปิดล่าสุด เช่น ')', ต้อง **match** กับสัญลักษณ์เปิดล่าสุด(*most recently seen*) '('.
- การจับคู่ดังกล่าวสามารถทำได้โดยใช้ stack.



Stack Application: Balanced Symbols (2)

Algorithm:

- Make an empty stack
- Read symbols:
 - If symbol is an opening symbol, **push** onto stack.
 - If symbol is a closing symbol:
 - If stack is empty, *report error!*
 - Else, (**top** and) **pop** stack. If popped symbol is not a matching opening symbol, *report error!*
 - At the end, if the stack is not empty, *report error!*

Stack Application: Simple Calculator

- กำหนดสมการทางคณิตศาสตร์ *arithmetic expression* ต้องการคำนวณค่าของสมการทำอย่างไรดี
- โดยทั่วไป จะใช้ *infix expressions* ในรูปของ
 $\text{operand1 OPERATOR operand2}$
- เช่น:
 - $1 + 2$
 - $4 * 3$
- การคำนวณ infix expressions ต้องคำนวณตามลำดับความสำคัญของเครื่องหมาย:
 - $1 + 2 * 3$
 - $10 - 4 - 3$
 - $2 \wedge 3 \wedge 3$
 - $1 - 2 - 4 \wedge 5 * 3 * 6 / 7 \wedge 2 \wedge 2$
- หรือใช้วงเล็บเป็นตัวจัดลำดับการคำนวณ: $(1-2)-((((4\wedge 5)*3)*6)/(7\wedge(2\wedge 2)))$

Stack Application: Postfix Notation

- โดยทั่วไป จะใช้ *postfix expressions* ในรูป
operand1 operand2 OPERATOR
- เช่น:
 - 1 2 +
 - 4 3 *
- With postfix expressions, precedence is **unambiguous**:
 - $1 - 2 - 4 \wedge 5 * 3 * 6 / 7 \wedge 2 \wedge 2$, or
 - $(1-2)-((((4^5)*3)*6)/(7^(2^2)))$, becomes
 - 1 2 - 4 5 ^ 3 * 6 * 7 2 2 ^ ^ / -

Stack Application: Postfix Notation (2)

- A **postfix machine** evaluates a postfix expression:
 - If an operand is seen, **push** it onto stack
 - If an operator is seen, **pop** appropriate number of operands, **evaluate** operator, **push** result onto stack
- When the complete postfix expression is evaluated, the stack should contain exactly one item: the **result**.

Stack Application: Infix to Postfix conversion

	Infix	Stack	Postfix
(a)	A+B*C-D/E		
(b)	+B*C-D/E		A
(c)	B*C-D/E	+	A
(d)	*C-D/E	+	AB
(e)	C-D/E	* +	AB
(f)	-D/E	* +	ABC
(g)	D/E	-	ABC*+
(h)	/E	-	ABC*+D
(i)	E	/ -	ABC*+D
(j)		/ -	ABC*+DE
(k)			ABC*+DE/-

Stack Application: Postfix expression evaluation

